ÓBUDAI EGYETEM

[Kar neve, pl. Neumann János Informatikai Kar]
[Intézet neve]

TUDOMÁNYOS DIÁKKÖRI DOLGOZAT

Vizsgakérdés-generálás helyben futtatható nyelvi modellekkel

A MIMIR *tervezésvezérelt, forráshivatkozásos RAG-módszere és kísérleti kiértékelése*

Szerzők:

[Szerző 1 neve]
[szak, évfolyam]

[Szerző 2 neve]
[szak, évfolyam]

Konzulens:

[Konzulens neve]
[beosztás, intézet]

Budapest, 2026

64. Tudományos Diákköri Konferencia – 2026. őszi félév

Tartalmi összefoglaló

A vizsgakérdések írása lassú, hibalehetőségekkel teli munka: a megadott helyes válasznak a tananyagból kell következnie, a rossz válaszoknak (disztraktoroknak) pedig hihetőnek, de igazolhatóan hibásnak kell lenniük. A nagy nyelvi modellek (LLM) megjelenésével elterjedt gyakorlat, hogy az oktató egy felhőben futó csevegő asszisztenstől kér kérdéseket. Ez azonban nem ellenőrzi a válaszokat, nem szabályozza a vizsga szerkezetét, és a tananyagot egy harmadik félnek adja át.

Dolgozatunkban bemutatjuk a MIMIR rendszert, amely az egyetlen kérést ellenőrizhető lépésekre bontott módszerrel váltja fel, és fogyasztói GPU-n, helyben futó, kvantált nyelvi modellekkel működik. A módszer a dokumentum szemantikus darabolása után vizsgatervet készít a kérdések témájáról és kognitív (Bloom-) szintjéről, kérdésenként keres a dokumentumban, forráshivatkozással ellátott kérdéseket ír, végül egy ellenőrző lánccal vizsgálja, hogy a forrás alátámasztja-e a helyes választ, és hogy a disztraktorok valóban hibásak-e. A rendszer a tananyagot csak a feladat idejére tárolja, a végső döntést pedig az oktatóra bízza.

A módszert egy nyílt kiértékelő keretrendszerrel vizsgáltuk: 14, egyszerre egy tényezőt változtató konfiguráció, egy független, bíróként működő nyelvi modell és dokumentumszintű statisztika. Egy pilot feltárta, hogy az ellenőrző visszacsatolása a helyes választ a kérdés szövegébe szivárogtathatja. A 30 dokumentumos fő vizsgálatban két helyi modellel (PULI-LlumiX 8B és Gemma 3 12B) a vizsgaterv mindkét modellnél szignifikánsan javított a naiv visszakereséses megoldáshoz képest ($+0{,}26$, illetve $+0{,}23$ a minőségi indexen). A módszer haszna a modell képességével csökkent: a gyengébb modelleknél nagy, egy erős 12B modellnél a teljes dokumentumot egy promptban kapó megoldás is hasonlóan jó. Egy hat dokumentumos előzetes vizsgálatban a 8 GB-os kártyán futó Qwen2.5-7B a módszerrel elérte a nagy szervermodell minőségét (0,96, illetve 0,95). A magyarra továbbtanított modell a formátumkövetésben bukott el, így a modellválasztás bármely komponensnél nagyobb hatásúnak bizonyult. A bíró oktatói validálása folyamatban van.

Kulcsszavak: automatikus kérdésgenerálás, visszakereséssel kiegészített generálás (RAG), kis nyelvi modellek, ellenőrzés, nyelvi modell mint bíró, adatvédelem, oktatástechnológia, magyar nyelv

Tartalomjegyzék

Tartalmi összefoglaló ii

Tartalomjegyzék iii

Rövidítések és fogalmak v

1. Bevezetés 1
1.1. Motiváció . . . 1
1.2. Problémafelvetés . . . 1
1.3. Kutatási kérdések és hipotézisek . . . 2
1.4. Hozzájárulások . . . 2
1.5. A dolgozat felépítése . . . 3

2. Szakirodalmi áttekintés 4
2.1. A jó feleletválasztós kérdés és a Bloom-taxonómia . . . 4
2.2. Automatikus kérdésgenerálás: a szabályoktól a nagy nyelvi modellekig . . . 5
2.3. Nagy nyelvi modellek röviden . . . 6
2.4. Visszakereséssel kiegészített generálás . . . 7
2.5. Feladatfelbontás és ellenőrzés . . . 8
2.6. Kis nyelvi modellek helyi futtatása . . . 10
2.7. A kiértékelés: nyelvi modell mint bíró és a mérés megbízhatósága . . . 10
2.8. Adatvédelem, szabályozás és a magyar nyelv . . . 11
2.9. Összegzés és a kutatási rés . . . 12

3. A MIMIR rendszer és generálási módszere 13
3.1. Tervezési célok . . . 13
3.2. Architektúra és megvalósítás . . . 13
3.3. Adatvédelem: zéró adatmegőrzés . . . 15
3.4. A GPU-memória becslése . . . 15
3.5. A generálási módszer . . . 16
3.5.1. Szemantikus darabolás . . . 17
3.5.2. Indexelés és az alapvonalak . . . 17
3.5.3. A vizsgaterv . . . 18
3.5.4. Kérdéshelyenkénti keresés . . . 18
3.5.5. Forráshoz kötött kérdésgenerálás . . . 19
3.5.6. Az ellenőrző lánc . . . 19
3.5.7. Munkamenet-hatókörű fogalomgráf . . . 21
3.5.8. Összeállítás és emberi felügyelet . . . 21

4. Kiértékelési módszertan 22
4.1. A tesztelés menete . . . 22
4.2. Adatkészlet . . . 22
4.3. A kísérleti konfigurációk . . . 23
4.4. A három vizsgálat . . . 25
4.5. Mutatók . . . 26
4.6. A bíró . . . 27

4.7. Oktatói értékelés . . . 27
4.8. Statisztikai elemzés . . . 28

5. Eredmények **30**
5.1. A pilot: három tanulság . . . 30
5.2. Az előzetes vizsgálat: Qwen2.5-7B és a szervermodell . . . 31
5.3. A fő vizsgálat: összesített eredmények . . . 32
5.4. A komponensek hatása . . . 33
5.5. Robusztusság: a naiv RAG törékenysége . . . 35
5.6. Miért bukik el a PULI-LlumiX? . . . 36
5.7. Az ellenőrző működése . . . 37
5.8. Lefedettség, kognitív szint és felhasználhatóság . . . 37
5.9. Erőforrásigény . . . 38
5.10. A mérés megbízhatósága . . . 39

6. Az eredmények értékelése **40**
6.1. A hipotézisek értékelése . . . 40
6.2. Mikor éri meg a módszer? . . . 41
6.3. Mit tanultunk az ellenőrzésről és a mérésről? . . . 42
6.4. Gyakorlati ajánlás . . . 42
6.5. Az érvényességet veszélyeztető tényezők . . . 43

7. Összefoglalás és további munka **44**
7.1. Összefoglalás . . . 44
7.2. További munka . . . 44

Irodalomjegyzék **46**

Ábrajegyzék **52**

Táblázatjegyzék **53**

A Promptok **55**
A.1. A naiv RAG (E0) és a teljes dokumentumos prompt (B-doc) . . . 55
A.2. A tervező . . . 55
A.3. A generátor . . . 55
A.4. A javító hívás . . . 56
A.5. Az ellenőrző . . . 56
A.6. A fogalomgráf (E3) . . . 57

B A bíró és az oktatók rubrikája **58**

C Paraméterek és reprodukálás **59**
C.1. A generálási folyamat paraméterei . . . 59
C.2. A reprodukálás lépései . . . 60

Rövidítések és fogalmak

BM25	szógyakoriságon alapuló lexikális rangsoroló függvény
GGUF, Q4_K_M	a `llama.cpp` kvantált modellformátuma, illetve annak kb. 4,5 bit/súly változata
GPU, VRAM	grafikus feldolgozóegység, illetve annak dedikált memóriája
KV-gyorsítótár	a szövegelőállítás közben tárolt kulcs- és értékvektorok
LLM	*Large Language Model*, nagy nyelvi modell
MCQ	*Multiple-Choice Question*, feleletválasztós kérdés
RAG	*Retrieval-Augmented Generation*, visszakereséssel kiegészített generálás
RRF	*Reciprocal Rank Fusion*, reciprok rangfúzió
bíró	a kérdéseket pontozó, a generátoroktól független nyelvi modell (gpt-oss-120b)
darab	a dokumentum egy összefüggő, néhány mondatos részlete, a keresés egysége
disztraktor	a feleletválasztós kérdés helytelen, de hihető válaszlehetősége
ellenőrző	a MIMIR saját ellenőrző lépése, amely a generált kérdést elfogadja vagy visszadobja
kérdéshely	a vizsgaterv egy pontja (fogalom, típus, Bloom-szint), amelyből egy kérdés készül
konfiguráció	a kísérlet egy beállítása (`E0`, `E1`, …), amely a referenciájától egy tényezőben tér el
kulcs	a feleletválasztós kérdés megjelölt helyes válasza
szivárgás	a kulcs szinte szó szerint szerepel a kérdés törzsében
vizsgaterv	a vizsga előre rögzített szerkezete kérdéshelyenként
F, G, A, V	formai megfelelés, alátámasztottság, vak megoldhatóság, disztraktor-validitás
Q	minőségi index, $(F+G+A+V)/4$
κ, α	Cohen-féle egyezési, illetve Krippendorff-féle megbízhatósági együttható
r_{rb}, $\tilde{p}$	rangbiszeriális hatásméret, illetve Holm-korrigált p-érték

1. Bevezetés

1.1. Motiváció

A feleletválasztós kérdés (*multiple-choice question*, MCQ) a tudásmérés egyik legszélesebb körben használt eszköze: gépi úton javítható, nagy létszámú kurzusokon is skálázható, és jól illeszkedik a tanulásmenedzsment-rendszerek (például a Moodle) kvízmoduljaihoz. Előállítása azonban drága. Egy jó kérdés törzse önmagában is érthető és egyértelmű, pontosan egy válaszlehetősége helyes a tananyag szerint, a helytelen válaszok (*disztraktorok*) pedig hihetők, de igazolhatóan hibásak [1, 2]. Egy tapasztalt oktató egy-egy ilyen kérdésen perceket, egy teljes vizsgán órákat dolgozik.

A nagy nyelvi modellek (*large language model*, LLM) megjelenése óta a legelterjedtebb megoldás a legegyszerűbb: az oktató feltölti a jegyzetét egy felhőben futó csevegő asszisztensbe, és „tíz feleletválasztós kérdést” kér. A modellek képesek kérdéseket és disztraktorokat fogalmazni [3], ugyanakkor hajlamosak folyékony, de a forrás által nem alátámasztott állításokat generálni [4]. Ezt a munkafolyamatot a továbbiakban *közvetlen megközelítésnek* nevezzük.

1.2. Problémafelvetés

A közvetlen megközelítés kényelmes, de az oktatási alkalmazás szempontjából négy gyengesége van.

(P1) **A helyesség nincs ellenőrizve.** Semmi sem garantálja, hogy a megjelölt helyes választ (a *kulcsot*) a dokumentum alátámasztja, és hogy a disztraktorok valóban hamisak; az önellenőrzés ezt csökkentheti, de nem szünteti meg [5].

(P2) **A szerkezet és a lefedettség nincs kontrollálva.** A kért darabszám, típus és nehézség csak kérés a promptban, és a hosszú bemenetet a modellek egyenetlenül használják [6], így a kérdések a dokumentum néhány részére tömörülhetnek.

(P3) **Adatvédelem.** A tananyag személyes adatot is tartalmazhat; a felhőszolgáltató az adatkezelés címzettjévé válik [7], és az intézmény elveszíti a kontrollt a tartalom felett.

(P4) **Függés a nagy, távoli modellektől.** A teljes dokumentum egyetlen promptban hosszú kontextust és nagy modellt igényel, a saját, fogyasztói hardveren futtatható kis modellek kontextusa és memóriája viszont korlátos.

A négy probléma nem oldható meg egyszerűen „jobb prompttal”. A P1 és a P2 a *módszer* hiányából fakad: ha a vizsga szerkezete, a kérdések forrása és az ellenőrzés nem önálló lépések, akkor sem mérni, sem kikényszeríteni nem lehet őket. A P3 és a P4 pedig egymást erősítik: a helyi futtatás megoldaná az adatvédelmi problémát, de a helyben futtatható kis modellek éppen a hosszú, egylépéses feladatban a leggyengébbek. Kutatásunk alapgondolata ezért az, hogy ha

a feladatot rövid, egyenként ellenőrizhető lépésekre bontjuk, egy fogyasztói grafikus kártyán (GPU) futó kis modell is képes lehet megbízható vizsgát előállítani, és ez a felbontás egyben a P1 és a P2 problémát is kezeli.

1.3. Kutatási kérdések és hipotézisek

RQ1 A strukturált generálás – tervezés, kérdésenkénti forráshoz kötés és ellenőrzés – érvényesebb és jobban lefedő vizsgát ad-e, mint ha a modellnek a dokumentumot (vagy annak visszakeresett részleteit) egyetlen promptban adjuk át?

RQ2 Mennyivel járulnak hozzá a minőséghez az egyes komponensek (tervező, ellenőrző, fogalomgráf, hibrid keresés, darabolás), és mennyibe kerülnek futási időben és modellhívásban?

RQ3 Mennyire közelíti meg egy fogyasztói GPU-n (8–12 GB) futó, 4 bitre kvantált kis modell egy nagy, szerveren futó modell teljesítményét, és hogyan függ az eredmény a választott helyi modelltől?

A kutatási kérdésekhez a következő hipotéziseket rendeljük; értékelésüket a 6.1. szakasz adja. A hipotézisekben szereplő konfigurációk (E0, E1, …) a kísérlet egy-egy beállítását jelölik (4.3. szakasz).

1. Hipotézis (Tervezés)**.** Azonos generátormodell mellett a vizsgaterv (E1) a naiv visszakereséses alapvonalhoz (E0) képest növeli a kérdések alátámasztottságát és a vizsga lefedettségét, a formai megfelelés romlása nélkül.

2. Hipotézis (Ellenőrzés)**.** Az ellenőrző lánc (E2) a tervezéshez (E1) képest tovább növeli az alátámasztottságot és a vak megoldhatóságot; az ellenőrző ítélete pozitívan egyezik a független bíró ítéletével ($\kappa > 0$).

3. Hipotézis (Lefedettség)**.** A teljes dokumentumot egyetlen promptban kapó alapvonal lefedettsége – modellmérettől függetlenül – alacsonyabb, mint a tervezett változatoké.

4. Hipotézis (Helyi vs. szerver)**.** A helyi kis modell a tervezett és ellenőrzött folyamatban a szervermodell naiv változatának minőségi indexét megközelíti (különbség $< 0{,}1$), miközben fogyasztói GPU-n fut.

5. Hipotézis (Mérés)**.** A generátoroktól eltérő modellcsaládba tartozó nyelvi modell (bíró) ítéletei az oktatói értékelésekkel legalább elfogadható megbízhatósággal egyeznek (Krippendorff-$\alpha \geq$ $\geq 0{,}667$).

1.4. Hozzájárulások

(1) **A MIMIR generálási módszere** (3. fejezet): szemantikus darabolás; lefedettségvezérelt vizsgaterv a Bloom-szintek arányos kiosztásával; kérdésenkénti, fogalomalapú keresés;

forrásrészletekre hivatkozó, egyenkénti kérdésgenerálás; a legolcsóbbtól a legdrágábbig rendezett ellenőrző lánc visszacsatolással.

(2) **Adatvédelmet biztosító helyi architektúra** (3. fejezet): lazán csatolt mikroszolgáltatások, zéró adatmegőrzés, ujjlenyomat-alapú audit, saját export (JSON, Moodle XML, PDF) és a GPU-memóriaigény mérésekkel ellenőrzött becslése.

(3) **Nyílt kiértékelési keretrendszer** (4. fejezet): 14 egytényezős kísérleti konfiguráció, kétnyelvű adatkészlet, forrásalapú mutatók, független bíró, oktatói értékelési protokoll és dokumentumszintű statisztika.

(4) **Empirikus eredmények** (5. és 6. fejezet): három vizsgálat, köztük egy 30 dokumentumos fő vizsgálat két helyi modellel. Ezek szerint a vizsgaterv minden vizsgált modellnél szignifikánsan javít a naiv visszakereséshez képest; a módszer haszna annál nagyobb, minél gyengébb a modell; a magyarra továbbtanított modell a formátumkövetésben bukik el; és a mérés több ponton (válaszszivárgás, csendes tartalékágak) a valósnál jobb eredményt mutatott volna, ha nem ellenőrizzük.

1.5. A dolgozat felépítése

A 2. fejezet áttekinti a feleletválasztós kérdés követelményeit, a kérdésgenerálás fejlődését, a visszakereséssel kiegészített generálást, az ellenőrzést, a kis modellek helyi futtatását és a nyelvi modellel végzett kiértékelést, és megfogalmazza a kutatási rést. A 3. fejezet a rendszert és a generálási módszert, a 4. fejezet a kiértékelési módszertant mutatja be. Az 5. fejezet a három vizsgálat eredményeit közli, a 6. fejezet ezeket a hipotézisek szempontjából értékeli, a 7. fejezet pedig összefoglal, és kijelöli a további munkát. A függelék a promptokat, a rubrikát és a reprodukálás lépéseit tartalmazza.

2. Szakirodalmi áttekintés

Ebben a fejezetben azokat a kutatási eredményeket tekintjük át, amelyekre a MIMIR módszere és kiértékelése épül. Először azt nézzük meg, *mit* akarunk előállítani (a jó feleletválasztós kérdést), majd azt, *hogyan* (kérdésgenerálás, nyelvi modellek, visszakeresés, feladatfelbontás és ellenőrzés, helyi futtatás), végül azt, *hogyan mérjük* az eredményt. A fejezetet a jogi és nyelvi környezet, valamint a P1–P4 problémákhoz kötött kutatási rés zárja.

2.1. A jó feleletválasztós kérdés és a Bloom-taxonómia

A feleletválasztós kérdés (*multiple-choice question*, MCQ) három részből áll: a *törzsből* (a kérdés vagy a befejezendő állítás), a *kulcsból* (a helyes válasz) és a *disztraktorokból* (a helytelen, de hihető válaszlehetőségek). Népszerűségét az adja, hogy gépileg, gyorsan és objektíven javítható; ára az, hogy jó kérdést írni nehéz.

A kérdésírás szabályait Haladyna, Downing és Rodriguez [1] foglalta össze 31 irányelvben. A tartalmi irányelvek szerint egy kérdés egyetlen, fontos tanulási célt mérjen; a formaiak szerint a törzs önmagában is érthető legyen és kerülje a tagadást; a disztraktorokra vonatkozók szerint a helytelen opciók legyenek hihetők, egymást kizárók, és formailag ne különbözzenek feltűnően a kulcstól, nehogy „árulkodó jelet” adjanak. Hogy ezek nem esztétikai kérdések, azt Downing [8] egy orvosi vizsgán mutatta meg: a szabálysértő kérdések miatt a hallgatók 10–15 %-a került a „nem felelt meg” kategóriába, akik a hibás kérdések nélkül átmentek volna. Egy gépi rendszernél tehát nem elég, hogy a kérdések „jónak tűnjenek”: a szabályok betartását ellenőrizni és mérni kell.

A disztraktorok számáról Rodriguez [9] metaanalízise azt találta, hogy a legtöbb esetben három opció is elegendő, mert a negyedik és ötödik ritkán „működik”. Tarrant és munkatársai [10] szerint a disztraktorok jelentős része *nem funkcionáló*, vagyis a hallgatók kevesebb mint 5 %-a választja, Gierl és munkatársai [2] áttekintése pedig rámutat, hogy az ilyen disztraktor gyakorlatilag csökkenti az opciók számát, és növeli a találgatással elérhető pontszámot. A MIMIR a Moodle-ban megszokott négyopciós formát követi, de a disztraktorok *minőségét* – hamisak és témába vágók-e – külön mutatóval méri (4.5. szakasz).

A kérdések kognitív szintjének leírására Bloom taxonómiáját használjuk Anderson és Krathwohl átdolgozott változatában [11]. Ez hat, egymásra épülő szintet különít el: *emlékezés* (tények felidézése), *megértés* (magyarázat saját szavakkal), *alkalmazás* (ismert eljárás új helyzetben), *elemzés* (összefüggések felismerése), *értékelés* (ítélet kritériumok alapján) és *alkotás* (új egész létrehozása). Feleletválasztós kérdéssel az első öt szint mérhető. Scully [12] szerint magasabb rendű gondolkodás is mérhető így, ha a törzs új helyzetet ír le, a gyakorlatban mégis a tényfelidéző kérdések dominálnak, mert ezeket a legkönnyebb megírni. Ugyanez várható egy nyelvi modelltől is, ha maga dönti el, milyen kérdést ír. A MIMIR ezért a kért nehézséghez előre rögzített

Bloom-szint-eloszlást rendel, és azt a vizsgatervben kérdésenként kiosztja (3.5.3. szakasz).

A kérdések klasszikus *tételelemzése* (nehézség, megkülönböztető erő) valódi hallgatói válaszokat igényel, így a vizsga előállításakor nem végezhető el; a generált vizsgák ilyen vizsgálata a további munka része (7. fejezet).

2.2. Automatikus kérdésgenerálás: a szabályoktól a nagy nyelvi modellekig

Az automatikus kérdésgenerálás (*automatic question generation*, AQG) több évtizedes terület; Kurdi és munkatársai [3] szisztematikus áttekintése 93 oktatási célú rendszert elemzett. A fejlődésben három generáció különíthető el.

Szabály- és sablonalapú rendszerek. Az első generáció a szöveget nyelvtanilag elemzi, megkeresi a „kérdezhető” mondatrészeket, és transzformációs szabályokkal kérdő mondattá alakítja őket. Mitkov és Ha rendszere [13] a fontos főnévi csoportokat választja kulcsnak, és egy lexikai adatbázis rokon fogalmait ajánlja disztraktornak. Heilman és Smith [14] a *túlgenerálás és rangsorolás* elvét vezette be: a szabályok sok jelöltet adnak, amelyek közül egy statisztikai modell választja ki a legjobbakat. Ezek a rendszerek átláthatók, és a kulcs mindig a szövegből származik, a kérdések viszont merevek, többnyire mondat-átfogalmazások, és minden nyelvhez külön szabályrendszer kell.

Neurális rendszerek. A második generációt a nagy, kézzel annotált adatkészletek tették lehetővé, például a Wikipédia-bekezdésekhez írt, több mint százezer kérdés–válasz párt tartalmazó SQuAD [15]. Du és munkatársai [16] ezeken tanítottak kódoló–dekódoló hálózatot, amely a bekezdésből és a kiválasztott válaszból szóról szóra állítja elő a kérdést. A kérdések folyékonyabbak, de a tanítóadat stílusát követik, és semmi sem garantálja, hogy a forrásból megválaszolhatók.

A *disztraktorgenerálás* külön irány; Alhazmi és munkatársai [17] újabb áttekintése a tudásbázis-alapú, a korpuszalapú és a generatív módszereket veti össze. Megállapításuk szerint a generatív módszerek adják a legtermészetesebb disztraktorokat, de gyakori hibájuk, hogy egy disztraktor valójában szintén helyes, vagy nyilvánvalóan abszurd. Pontosan ezt a két hibatípust vizsgálja disztraktoronként a MIMIR ellenőrzője és a bíró.

Nagy nyelvi modellek. A harmadik generációban a kérdéseket egy nagy nyelvi modell írja utasítás alapján. Wang és munkatársai [18] GPT-3-mal generáltak tankönyvi kérdéseket, amelyek egy része emberi értékelés szerint az emberek által írtakkal összemérhető volt. Doughty és munkatársai [19] programozási kurzusokon vetették össze a GPT-4 és az oktatók kérdéseit: a gépi kérdések többsége jól megfogalmazott volt, kisebb részükben azonban a kulcs vagy egy disztraktor hibás volt – ezek átnézés nélkül a vizsgába kerültek volna. Bitew és munkatársai [20] a promptba tett, már jóváhagyott példakérdésekkel javították a disztraktorokat; ugyanez a csoport készítette az OpenStax-tankönyvekhez írt, szakértők által ellenőrzött EduQG adatkészletet [21], amelynek angol dokumentumait mi is használjuk. A kis modellek oldaláról Fawzi és munkatársai [22] kimutatták, hogy egy kis modell oktatási szövegeken végzett további tanítással a

nagy modellekhez mérhető kérdéseket írhat. Ezek a munkák a kész kérdéseket utólag, emberi értékeléssel mérik; arra, hogy a generálás *folyamata* hogyan biztosíthatja a helyességet és a lefedettséget, kevesebb figyelem jut. Ez az a pont, ahol a MIMIR hozzájárulni kíván.

A kiértékelés problémája. Mulla és Gharpure [23] szerint a rendszerek jelentős részét a gépi fordításból átvett, referencia-átfedésen alapuló metrikákkal (BLEU, ROUGE) mérik, amelyek azt számolják, hány szó egyezik a generált és egy ember írta referenciakérdés között. Egy vizsgakérdés azonban egyetlen egyező szó nélkül is lehet kiváló, hibás kulccsal pedig használhatatlan. Kurdi és munkatársai [3] azt is kimutatták, hogy a humán értékelések jellemzően kis mintán készülnek, és ritkán ellenőrzik a kulcs helyességét. Munkánkban ezért a helyességet a forrással szembeni ítéletekkel mérjük, a referenciakérdéseket pedig csak segédmutatókhoz használjuk (4. fejezet).

2.3. Nagy nyelvi modellek röviden

A mai nyelvi modellek alapja a transzformer [24], amely a szöveget *tokenekre* (szavakra vagy szódarabokra) bontja, és minden token belső reprezentációját a többi token függvényében számolja ki. Két felhasználási módja alakult ki. A *kódoló* modellek (például a BERT [25]) a teljes szöveget egyszerre látják, és jelentését egy vektorban (*beágyazásban*) foglalják össze; ezeket keresésre és osztályozásra használják. A *dekódoló* vagy generatív modellek (például a GPT-család [26]) a szöveget balról jobbra, tokenenként állítják elő. A csevegő asszisztenseket ezen felül utasításkövetésre is hangolják. A MIMIR mindkét típust használja: kódoló modellt a darabolásra és a keresésre, generatív modellt a kérdések megírására.

A szótár szódarabokból áll: a gyakori szavak egy, a ritkák több tokent kapnak. A toldalékoló magyar szavak ezért átlagosan több tokenre bomlanak, mint az angolok, így ugyanaz a tartalom magyarul több helyet foglal a modell bemenetében (*kontextusában*). Két szöveg jelentésbeli hasonlóságát a beágyazásaik által bezárt szög koszinuszával mérjük:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}, \tag{2.1}$$

amelynek értéke 1, ha a két vektor azonos irányú, és annál kisebb, minél jobban eltér a két szöveg jelentése. A dolgozatban a darabolás, a keresés, a duplikátumszűrés és a lefedettség mérése is erre a mértékre épül.

Hallucináció. A nyelvi modellek legfontosabb gyengesége a *hallucináció*: folyékony, magabiztos, de a forrással vagy a valósággal ellentétes szöveg előállítása. Ji és munkatársai [4] két típust különítenek el. Az *intrinzikus* hallucináció ellentmond a forrásnak (például a forrás szerint a kávé a 17. században jutott el Európába, a modell a 9. századot írja), az *extrinzikus* pedig olyan állítás, amely a forrásból sem nem igazolható, sem nem cáfolható. A kérdésgenerálásban mindkét típus súlyos: ha a kulcs ellentmond a forrásnak, a felkészült hallgató veszít pontot;

ha a forrásban nem szerepel, a kérdés a tananyagból nem válaszolható meg. Ezért tekintjük a legfontosabb mutatónak az *alátámasztottságot*, vagyis azt, hogy a forrás igazolja-e a kulcsot.

Strukturált kimenet. A generátornak gépileg feldolgozható JSON-kimenetet kell adnia, de egy kis modell időnként hibás JSON-t ír. Ezt vagy utólagos ellenőrzéssel és újrakéréssel, vagy *korlátozott dekódolással* lehet kezelni, amely generálás közben csak a formátumnak megfelelő tokeneket engedi meg [27]. A MIMIR az előbbit alkalmazza: a formai hibát az ellenőrző első lépése szűri.

2.4. Visszakereséssel kiegészített generálás

A hallucináció elleni első stratégia, hogy a modell ne csak a paramétereiben tárolt tudásra támaszkodjon. A visszakereséssel kiegészített generálás (*retrieval-augmented generation*, RAG) során a rendszer a válasz előtt releváns szövegrészleteket keres egy gyűjteményből, és ezeket a promptba téve kéri a választ. A RAG nevet Lewis és munkatársai [28] vezették be, akik egy sűrű keresőt és egy generatív modellt kapcsoltak össze.

Gao és munkatársai [29] áttekintése három paradigmát különít el. A *naiv RAG* egyetlen keresést és egyetlen generálást végez: a lekérdezéshez legjobban hasonló néhány részletet a promptba másolja. A *fejlett RAG* a keresés előtt és után is javít (például átírja a lekérdezést vagy újrarangsorolja a találatokat). A *moduláris RAG* a keresési, generálási és ellenőrzési lépéseket szabadon kombinálja. A MIMIR tervezett generálási folyamata ebben a szóhasználatban moduláris RAG: a keresés a vizsgaterv minden pontjára külön fut, és az ellenőrzés eredménye visszahat rá.

Lexikális és sűrű keresés. A *lexikális* keresés szavak alapján indexel; legismertebb képviselője a BM25 [30], amely azokat a részleteket rangsorolja előre, amelyekben a lekérdezés szavai gyakoriak, de a gyűjteményben ritkák. Gyors, nem igényel tanítást, és jól kezeli a szakkifejezéseket és tulajdonneveket, de a szinonimákat és a magyar ragozott alakokat nem ismeri fel. A *sűrű* keresés a szövegeket egy közös vektortérbe képezi, és a hasonlóságot (2.1) szerinti koszinusszal méri. A DPR [31] megmutatta, hogy ez a nyílt kérdésmegválaszolásban felülmúlhatja a BM25-öt. Az általunk használt többnyelvű E5 modell [32] nagy szövegpárgyűjteményen, kontrasztív tanítással készült, és a magyart is támogatja. A keresők összehasonlító mérései szerint azonban egyik módszer sem a legjobb minden adatkészleten [33]. Ezért gyakori a *hibrid* keresés, amely a két rangsort a reciprok rangfúzióval (RRF) [34] egyesíti: egy részlet annál több pontot kap, minél előrébb áll az egyes rangsorokban. A MIMIR egyik konfigurációja (`E2h`) ezt a változatot vizsgálja (4.3. szakasz).

Lekérdezés-átírás. A keresés minősége a lekérdezésen is múlik: ha az rövid, általános vagy más stílusú, mint a keresett szöveg, a legjobb kereső is rossz találatot ad. A HyDE [35] ezért egy nyelvi modellel előbb egy „hipotetikus” választ írat, és annak beágyazásával keres, mert az jobban hasonlít a keresett szövegre, mint maga a kérdés. Vizsgagenerálásban ez kulcskérdés. A

naiv RAG lekérdezése ugyanis egy *feladatleírás* („10 feleletválasztós kérdés, könnyű”), amely semmit sem mond a keresett tartalomról. A MIMIR vizsgaterve a tervező által javasolt tartalmi fogalmakkal keres, ami lényegében lekérdezés-átírás. Eredményeink szerint ez a különbség áll a naiv RAG leggyakoribb kudarca mögött (5. fejezet).

Darabolás. A keresés egysége a *darab*: a dokumentumot rövidebb részletekre kell bontani. A rögzített hosszúságú ablak mondatok közepén is vághat; a *szemantikus* darabolás ott vág, ahol a szomszédos mondatok beágyazásainak hasonlósága hirtelen lecsökken, vagyis valószínűleg témaváltás van. Qu, Tu és Bao [36] szerint a szemantikus darabolás előnye adatkészletfüggő és gyakran kisebb a vártnál. Ezért a darabolási stratégiát külön konfigurációkkal (E5a–E5c) vizsgáljuk, ahelyett hogy eleve jobbnak tekintenénk a szemantikus változatot (3.5.1. szakasz). Eredményeink szerint a szemantikus darabolás rövid, tartalom nélküli darabokat is létrehozhat, amelyekhez éppen a feladatleírás típusú lekérdezés áll a legközelebb.

Forráshivatkozás. A visszakeresés nem garantálja, hogy a válasz a visszakeresett szövegen alapul. Ezért egyre több rendszer kéri, hogy a modell *hivatkozzon* a felhasznált részletre. Az ALCE benchmark [37] azt méri, hogy a hivatkozások valóban alátámasztják-e az állításokat, és még a legjobb modelleknél is sok hiányos vagy pontatlan hivatkozást talált. Rashkin és munkatársai [38] szerint egy állítás akkor *tulajdonítható* egy forrásnak, ha egy átlagos olvasó a forrás alapján egyetértene vele. A MIMIR minden kérdéshez megköveteli a kulcsot igazoló darabok azonosítóját, és a bíró is ehhez hasonló, olvasóközpontú definícióval dönt az alátámasztottságról.

Gráfalapú keresés és hosszú kontextus. A GraphRAG [39] a gyűjteményből nyelvi modellel fogalom- és kapcsolatgráfot épít, és ennek csoportjai alapján válaszol; a MIMIR ennek egy könnyített, egyetlen dokumentumra korlátozott változatát vizsgálja (E3, 3.5.7. szakasz). A RAG valódi alternatívája az, hogy a teljes dokumentumot egyetlen promptban adjuk át, ahogy az oktatók a gyakorlatban teszik. A névleges kontextushossz viszont nem azonos a ténylegesen kihasznált hosszal: Liu és munkatársai [6] szerint a modellek a bemenet elején és végén lévő információt jobban használják, mint a közepén lévőt, a RULER benchmark [40] szerint pedig a teljesítmény a hossz növelésével folyamatosan romlik. Li és munkatársai [41] összevetésében elegendő erőforrás mellett a hosszú kontextus átlagosan jobb, a RAG viszont nagyságrenddel olcsóbb, és a kérdések jelentős részén azonos eredményt ad. Ebből két várakozás következik: a teljes dokumentumot kapó modell kérdései a dokumentum egyes részeire tömörülhetnek (3. hipotézis), és a hosszú kontextus egy kis kártyán a memóriaigény miatt korlátos (3.4. szakasz). Ugyanakkor egy erős modellnél a teljes dokumentumos prompt komoly versenytárs, ezért ezt is mérjük.

2.5. Feladatfelbontás és ellenőrzés

A dolgozat alapgondolata, hogy egy összetett feladatot rövid, jól körülhatárolt lépésekre bontva egy kisebb modell is megbízhatóbban teljesít. Wei és munkatársai [42] szerint a modell összetett

feladatokban pontosabb, ha lépésenkénti gondolatmenetet ír (*chain-of-thought*). A *least-to-most* módszer [43] a problémát előbb részfeladatokra bontja, majd ezeket egymás után oldja meg, a *Plan-and-Solve* [44] pedig arra kéri a modellt, hogy előbb tervet készítsen, és csak utána hajtsa végre – ez a vizsgaterv gondolatának közvetlen megfelelője. Ezek a módszerek a felbontást a promptra bízzák. Wu, Terry és Cai [45] *MI-láncai* ezzel szemben a programban kötik össze a nyelvi modell egymásra épülő hívásait. Felhasználói vizsgálatuk szerint a láncok nemcsak jobb eredményt adtak, hanem *átláthatóbbak* és jobban *irányíthatók* is voltak, mert minden köztes lépés látható és módosítható.

A MIMIR a második irányba tartozik: a szerkezeti döntéseket (hány kérdés, melyik témából, milyen szinten) a program hozza meg, a nyelvi modell csak jól körülhatárolt részfeladatot végez (fogalmak javaslása, egy kérdés megírása, egy ítélet). Így minden lépés rövid kontextussal dolgozik, ami egy kis modellnek kedvez; minden köztes eredmény visszakövethető; és a lépések egyenként cserélhetők, ami lehetővé teszi az egytényezős kísérleti tervet. Az ár a több modellhívás és a hosszabb futási idő.

Ellenőrzés és önjavítás. A hallucináció elleni második stratégia a kimenet utólagos ellenőrzése. A Self-Refine [46] a modellel értékelteti és javíttatja saját kimenetét, a Chain-of-Verification [47] ellenőrző kérdéseket tesz fel a saját válaszáról, és az eltérések alapján javít. A FActScore [48] a szöveget atomi tényekre bontja, és mindegyiket külön veti össze egy tudásforrással. Egy másik ág a *természetes nyelvi következtetés* (*natural language inference*, NLI): annak eldöntése, hogy egy állítás következik-e egy szövegből, ellentmond-e neki, vagy egyik sem. Az ilyen modellek az összefoglalók tényszerűségének ellenőrzésére is alkalmasak [49]. Az alátámasztottság eldöntése pontosan ilyen feladat: következik-e a forrásból, hogy a megjelölt kulcs a helyes válasz?

Az önellenőrzésnek azonban korlátai vannak. Huang és munkatársai [5] szerint *külső visszajelzés nélkül* az önjavítás érvelési feladatokban nem javít, sőt ronthat is. Snell és munkatársai [50] ugyanakkor kimutatták, hogy a futás közben elköltött többletszámítás (több minta, ellenőrzés, újrapróbálkozás) bizonyos feladatokban helyettesítheti a nagyobb modellt. A két eredmény együtt azt sugallja, hogy az ellenőrzés akkor hasznos, ha *külső* információt visz a folyamatba: determinisztikus szabályokat, a forrásszöveget vagy egy másik modell ítéletét. A MIMIR ellenőrzője ezért olcsó szabályokat (forma, hivatkozás, szivárgás, duplikátum) kombinál a forrással szembeni, nyelvi modellel végzett ítéletekkel, és a hibát visszajelzésként adja vissza a generátornak (3.5.6. szakasz).

A visszacsatolásnak saját veszélye is van. Goodhart törvénye Strathern megfogalmazásában: ha egy mérőszám céllá válik, megszűnik jó mérőszám lenni [51]. Ha a generátort addig javíttatjuk, amíg át nem megy az ellenőrzésen, olyan megoldást is találhat, amely az ellenőrzést kielégíti, a valódi célt viszont nem. A pilot vizsgálatunk egy ilyen esetet tárt fel: a legolcsóbb út a forrásmondat szinte szó szerinti bemásolása volt a törzsbe (*szivárgás*, 5.1. szakasz).

2.6. Kis nyelvi modellek helyi futtatása

A helyi futtatás feltétele, hogy a modell elférjen egy fogyasztói grafikus kártya memóriájában. Ma több nyílt modellcsalád ér el jó teljesítményt néhány milliárd paraméterrel, például a Qwen2.5 [52], a Llama 3.1 [53] és a Gemma 3 [54]; ezek a kisebb méretet több és jobb minőségű tanítóadattal ellensúlyozzák. A pilotban és az előzetes vizsgálatban a Qwen2.5-7B, a fő vizsgálatban a Gemma 3 12B és a PULI-LlumiX 8B a generátor.

A magyar nyelvtechnológiai közösség saját modelleket is készített, például a huBERT kódolót [55] és a NYTK PULI modellcsaládját. Ennek legújabb tagja, a PULI-LlumiX [56] a Llama 3.1 8B modell magyar szövegeken folytatott további előtanításával készült. Természetes várakozás, hogy egy magyarra hangolt modell magyar tananyagból jobb kérdéseket ír, mint egy általános többnyelvű modell; ezt a várakozást a fő vizsgálatban közvetlenül teszteljük (5. fejezet).

A memóriaigény fő összetevője a súlyok mérete. Egy 7 milliárd paraméteres modell 16 bites ábrázolásban paraméterenként 2 bájtot, összesen 14 GB-ot foglal. A *kvantálás* a súlyokat kevesebb biten tárolja: a GPTQ [57] a tanítás után 3–4 bitre tömörít. A helyi futtatás de facto szabványa a `llama.cpp` [58] GGUF formátuma és csoportos K-kvantálása; az általunk használt Q4_K_M változat átlagosan kb. 4,5–5 bitet használ súlyonként, és erre épül az Ollama futtatókörnyezet [59] is. Átfogó kiértékelések szerint a 4 bites súlykvantálás általános feladatokon csak kis pontosságvesztést okoz [60]. A memóriaigény második összetevője a *kulcs–érték (KV) gyorsítótár*, amely a már feldolgozott tokenek köztes eredményeit tárolja, így a kontextus hosszával arányosan nő. Mindezek alapján egy 7–8B modell 4 bites változata egy 8 GB-os, a Gemma 3 12B egy 12 GB-os fogyasztói kártyán futtatható; a számszerű becslést a 3.4. szakasz adja meg.

2.7. A kiértékelés: nyelvi modell mint bíró és a mérés megbízhatósága

Az emberi értékelés drága és lassú, ezért elterjedt, hogy egy erős nyelvi modell pontozza a kimenetet (*LLM-as-a-judge*); a dolgozatban ezt a független, pontozó modellt *bírónak* nevezzük. Zheng és munkatársai [61] szerint egy erős bíró ítéletei közel annyira egyeznek az emberi preferenciákkal, mint az emberek ítéletei egymással. Chiang és Lee [62] hasonló eredményre jutott úgy, hogy a modell szó szerint ugyanazt az utasítást és értékelőlapot kapta, mint az emberi szakértők – ezt az elvet a mi rubrikánk is követi. A RAGAS [63] a RAG-rendszerek referencia nélküli, nyelvi modellel számolt hűség- és relevanciamutatóit javasolja.

A bírók torzításai jól dokumentáltak: a *pozíciós torzítás* az első vagy második helyen álló válasz előnyben részesítése [64], a *hossztorzítás* a hosszabb válaszoké, az *önpreferencia* pedig a saját modellcsalád szövegeié [65]. Ezért bíránk, a gpt-oss-120b [66] más modellcsaládba tartozik, mint bármelyik generátor (4.6. szakasz).

A bíró csak akkor érvényes mérőeszköz, ha ítéletei egyeznek az emberi szakértőkével. Két értékelő bináris ítéleteire Cohen-κ-t [67], több értékelőre és ordinális skálára Krippendorff-α-

t [68] használnak; mindkettő azt méri, mennyivel nagyobb az egyezés a véletlenül várhatónál. Landis és Koch [69] skálája szerint 0,20 alatt csekély, 0,21–0,40 között gyenge, 0,41–0,60 között közepes, 0,60 felett erős az egyezés; Krippendorff szerint $\alpha \geq 0{,}8$ megbízható, 0,667 és 0,8 között óvatos következtetésre elegendő. A bíró oktatói validálását a 4.7. szakasz írja le.

A kísérletek megbízhatósága. Bouthillier és munkatársai [70] kimutatták, hogy a gépi tanulási eredményekben a véletlen tényezők okozta szórás gyakran összemérhető a közölt javulásokkal, így egyetlen futás alapján nem szabad két módszer között dönteni. Dror és munkatársai [71] szerint a nyelvtechnológiai cikkek jelentős része nem végez szignifikanciatesztet, vagy az adatokhoz nem illő próbát használ; a 0 és 1 közé szorított, ferde eloszlású mutatóknál a nemparaméteres próbák (például a Wilcoxon-próba) és a bootstrap a biztonságosabbak. Fontos a mérés *egysége* is: egy dokumentum tíz kérdése nem tíz független megfigyelés, ezért mi a dokumentumot tekintjük egységnek, és a sok összevetésből adódó véletlen „szignifikanciát” Holm-korrekcióval [72] szűrjük (4.8. szakasz). Veszélyt jelent az *adatszennyezés* is [73]: a nyilvános Wikipédia- és OpenStax-szövegek a modellek tanítóadatában is szerepelhettek, így a bíró a kérdést a forrás nélkül, emlékezetből is megválaszolhatja.

Végül a mérés az oktatót is érinti. Parasuraman és Manzey [74] szerint a felhasználók hajlamosak túlságosan megbízni az automatikus rendszerek jelzéseiben (*automatizálási torzítás*): ha egy rendszer egy kérdést „ellenőrzöttnek” jelöl, az oktató kevésbé figyelmesen nézi át. Ezért az ellenőrző jelzéseit a MIMIR felületén figyelmeztetésként, nem ítéletként jelenítjük meg, amíg megbízhatóságukat mérések nem igazolják.

2.8. Adatvédelem, szabályozás és a magyar nyelv

Ha a tananyag személyes adatot tartalmaz (például esettanulmányban szereplő neveket vagy hallgatói munkákat), a felhőszolgáltató a GDPR [7] szerint adatkezelővé vagy adatfeldolgozóvá válik, az adattakarékosság elve szerint pedig csak a feltétlenül szükséges adat kezelhető. Ehhez technikai kockázat is társul: Carlini és munkatársai [75] kimutatták, hogy a tanítóadat egyes részletei a kész modellből szó szerint visszanyerhetők. Az EU MI-rendelete (AI Act) [76] az oktatásban használt rendszerek egy részét – például a tanulási eredmények értékelésére szolgálókat – magas kockázatúnak minősíti, és emberi felügyeletet vár el; az UNESCO útmutatója [77] ugyanezt fogalmazza meg oktatáspolitikai szinten. A MIMIR tervezési céljai – helyi futtatás, zéró adatmegőrzés, kötelező oktatói jóváhagyás – közvetlenül ezekből következnek (3.1. szakasz).

A magyar nyelv. A magyar két szempontból jelent külön kihívást. Agglutináló nyelv, így egy szótőnek sok alakja van („immunrendszer”, „immunrendszert”, „immunrendszerben”), ami a lexikális keresést nehezíti és a tokenszámot növeli. Emellett kis erőforrású nyelv: a MEGA kiértékelés [78] szerint a generatív modellek teljesítménye a kisebb erőforrású nyelveken jelentősen elmarad az angoltól, ezért egy kis modell magyar szövegében nyelvtani hibákra is számítani kell, amit a kiértékelésnek külön mérnie kell. A magyar nyelvértési feladatok egységes

2.1. táblázat. A kérdésgenerálási megközelítések összevetése a P1–P4 problémák szempontjából (✓: kezeli, ○: részben, –: nem kezeli). A táblázat azt mutatja, hogy egy megközelítés foglalkozik-e az adott szemponttal, nem azt, hogy milyen jól.

Megközelítés	Kulcs ellenőrzése	Szerkezet, lefedettség	Helyi futás	Kis modell	Forrás-hivatkozás	Helyesség mérése
Szabály/sablon alapú [13, 14]	○	–	✓	✓	○	–
Neurális seq2seq [16]	–	–	✓	✓	–	–
Dokumentum → csevegő LLM (status quo)	–	–	–	–	–	–
LLM-alapú kérdésgenerálás [18, 19]	–	○	–	–	–	○
Naiv RAG [28]	–	–	○	○	○	–
Továbbtanított kis modell [22]	–	–	✓	✓	–	○
Feladatfelbontó promptolás [43, 44]	–	○	–	–	–	–
Önjavítás / CoVe [46, 47]	○	–	–	–	–	–
Hivatkozó generálás [37]	○	–	–	–	✓	✓
GraphRAG [39]	–	○	–	–	○	–
MIMIR (jelen munka)	✓	✓	✓	✓	✓	○

mérésére a HuLU benchmark [79] szolgál, a MILQA [80] pedig magyar Wikipédia-cikkekre épülő kérdés–válasz adatbázis, amelyet adatforrásként használunk (4.2. szakasz). Tudomásunk szerint magyar nyelvű vizsgakérdés-generálásról még nem jelent meg rendszerszintű kiértékelés.

2.9. Összegzés és a kutatási rés

A 2.1. táblázat a tárgyalt megközelítéseket a bevezetés P1–P4 problémái szerint veti össze. A korai rendszerek helyben futnak, de sem a helyességet, sem a szerkezetet nem ellenőrzik; a csevegő modell egyik problémát sem kezeli; a naiv RAG forráshoz köti a generálást, de a feladatleírásból készült lekérdezéssel a dokumentum kis részét látja; az önjavító, a hivatkozó és a gráfalapú módszerek egy-egy problémát részben kezelnek, de nagy modellt feltételeznek. A MIMIR sorában a helyesség mérése csak részben teljesül: a bíró oktatói validálása még hátravan, és a saját ellenőrzőjének kérdésenkénti megbízhatósága is korlátozottnak bizonyult (5. fejezet).

Az összevetésből három hiányosság rajzolódik ki. (i) Nincs olyan nyílt rendszer, amely a kérdésgenerálást explicit tervvel, kérdésenkénti forráshivatkozással és ellenőrzéssel, fogyasztói GPU-n (8–12 GB) futó kis modellel végezné. (ii) A kiértékelések ritkán mérik a kulcs helyességét és a vizsga lefedettségét, és hiányoznak az egytényezős kísérleti tervek, amelyek a komponensek hatását egyenként választják szét. (iii) A magyar nyelvű vizsgakérdés-generálás kiértékelése feltáratlan. Az első hiányosságra a 3., a másodikra a 4., a harmadikra – a rendelkezésre álló adatok korlátai között – az 5. fejezet ad választ.

3. A MIMIR rendszer és generálási módszere

Ez a fejezet először a rendszer tervezési céljait, felépítését, adatvédelmi megoldásait és memóriaigényét mutatja be (3.1–3.4. szakasz), majd lépésenként ismerteti a generálási módszert (3.5. szakasz). A módszer leírása a kiértékelő keretrendszer implementációját követi (`tests/eval/mimir_eval/blueprint`); a paraméterek alapértékeit a C. függelék foglalja össze.

3.1. Tervezési célok

A bevezetés P1–P4 problémáiból és az AI Act emberi felügyeletre vonatkozó elvárásából [76] öt tervezési célt vezettünk le. A 3.1. táblázat ezeket a közvetlen megközelítéssel (a dokumentum feltöltése egy csevegő asszisztensbe) veti össze.

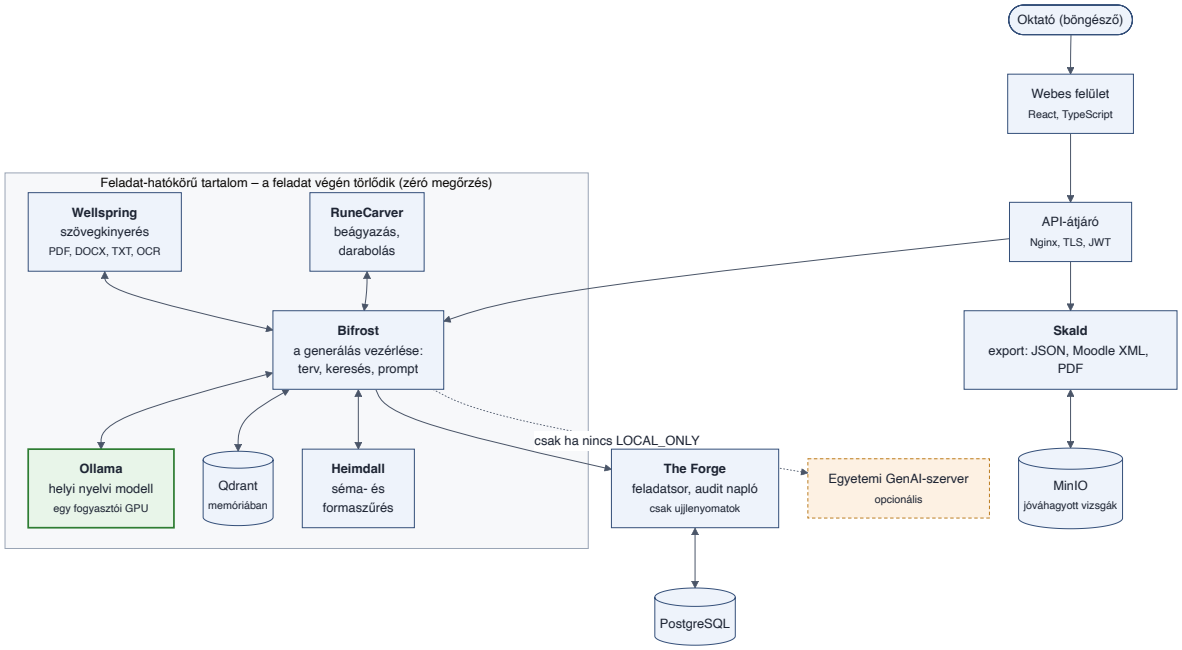
3.1. táblázat. Tervezési célok: a dokumentum csevegő nyelvi modellnek való átadása és a MIMIR összevetése.

Cél	Dokumentum → csevegő modell	MIMIR
G1 Ellenőrizhető	A kulcs nincs ellenőrizve, nincs forráshivatkozás	Minden kérdés a forrásrészletekre hivatkozik; alátámasztottsági és vak megoldási ellenőrzés; az ellenőrzésen át nem jutott kérdés megjelölve
G2 Kontrollált	Darabszám, típus, nehézség csak kérés	A vizsgaterv rögzíti a darabszámot, a típusokat és a Bloom-szinteket; lefedettségvezérelt fogalomválasztás
G3 Adatvédő	Az anyag a szolgáltatóhoz kerül	Helyi modell; a tartalom a feladat végén törlődik; kapcsolható, kizárólag helyi mód
G4 Megvalósítható	Hosszú kontextus, nagy modell	Rövid hívások 8 192 tokenes kontextusban; fogyasztói GPU (8–12 GB)
G5 Felügyelt	A kimenet változtatás nélkül használható	Kötelező oktatói jóváhagyás szerkesztőben, a hivatkozott forrással; MI-közreműködés jelölése az exportban

3.2. Architektúra és megvalósítás

A rendszer lazán csatolt, konténerizált szolgáltatásokból áll, amelyeket egy nginx-alapú API-átjáró fog össze (3.1. ábra). A szolgáltatások az északi mitológia tudással kapcsolatos alakjairól kapták nevüket: a *Wellspring* PDF-ből, DOCX-ből és szövegfájlból nyeri ki a szöveget (olvashatatlan PDF-oldalnál helyi OCR-rel); a *RuneCarver* mondatokra bont, mondatbeágyazást számol és darabol (3.5.1. szakasz); a *Bifrost* a generálás karmestere (feladatkezelés, indexelés, keresés,

promptépítés, modellválasztás); a *Heimdall* JSON-sémák és szabályok alapján szűri a modellválaszokat; a *The Forge* a feladatsort és az audit naplót kezeli; a *Skald* pedig JSON, Moodle XML és PDF formátumba exportál. Minden szolgáltatás FastAPI-alapú Python-alkalmazás egységes, verziózott felülettel, így a kiértékelő keretrendszer pontosan ugyanazokat a végpontokat hívja, mint az éles webes felület. A dokumentumfeldolgozás, a darabolás, a RAG-folyamat, a munkáskészlet és az export (a Moodle XML CDATA-blokkokkal és MathJax-támogatással, a PDF saját tördelőkóddal) saját implementáció, nem magas szintű keretrendszerek összekötése.



3.1. ábra. A MIMIR szolgáltatásai és adatfolyama. A feltöltött tartalmat a feldolgozó szolgáltatások csak a feladat élettartama alatt, memóriában tartják; a nyelvi modell az egyetlen GPU-t használó összetevő, külső modellt csak kikapcsolt `LOCAL_ONLY` kapcsoló mellett hív a rendszer.

A generálás percekig tarthat, ezért a rendszer *aszinkron*: a generálási kérés azonnal egy feladatazonosítót ad vissza, az állapot (haladás, becsült hátralévő idő) pedig egy másik végponton kérdezhető le. A háttérmunkát a The Forge egy adatbázistáblán alapuló sorral osztja ki, amely a PostgreSQL `FOR UPDATE SKIP LOCKED` záradékára [81] épül: minden munkás a legrégebbi, még senki által nem zárolt feladatot veszi ki, így egy feladatot pontosan egy munkás kap meg, külön üzenetközvetítő szoftver nélkül. Mivel a GPU egyszerre egy modellt futtat, a generálási feladatok egymás után futnak; a párhuzamos munkások a CPU-igényes kinyerést és darabolást gyorsítják.

A React és TypeScript alapú, magyar és angol nyelvű webes felületen az oktató feltölti a dokumentumot, megadja a kérdések számát, típusát (feleletválasztós, igaz-hamis, kifejtős) és nehézségét, majd a kész vázlatot a szerkesztőben (3.5.8. szakasz) átnézi és exportálja.

3.3. Adatvédelem: zéró adatmegőrzés

A G3 cél az adatminimalizálás és a tárolási korlátozás GDPR-elvein [7] alapul: a feltöltött tartalom csak addig létezik, amíg a feladathoz szükséges (3.2. táblázat).

3.2. táblázat. Adatfajták életciklusa a MIMIR rendszerben.

Adat	Tárolás	Élettartam
Fájl, kinyert szöveg, darabok, promptok	memória	a feladat (ill. a modellhívás) végéig
Beágyazások	Qdrant, memóriában	a következő dokumentum indexelése előtt törlődnek
Generált vizsga (vázlat)	feladattár	60 perc, majd automatikus törlés
Jóváhagyott, mentett vizsga	MinIO	beállítható megőrzés; az oktató törölheti
Audit napló	PostgreSQL	csak metaadatok és ujjlenyomatok

Az audit napló az elszámoltathatóság miatt fontos, de nem tartalmazhat tananyagot. Ezért a feladat azonosítóját, a modell nevét, a promptverziót és a minőségi pontszámot tárolja, a kérésből, a promptból és a visszakeresett kontextusból pedig csak a SHA-256 *ujjlenyomatot* (kriptográfiai kivonatot) és a szöveg hosszát. Az ujjlenyomattal utólag igazolható, hogy két futás ugyanazzal a kontextussal dolgozott-e, a tartalom azonban nem állítható vissza belőle. A `LOCAL_ONLY` kapcsoló minden külső modellhívást letilt: ilyenkor a rendszer kizárólag a helyi modellt és a helyi OCR-t használja.

3.4. A GPU-memória becslése

A G4 cél kvantitatív kérdés: elfér-e a generátor egy fogyasztói videokártyán úgy, hogy mellette az asztali környezet is fut? A szövegelőállítás közbeni memóriaigény három tag összege:

$$M \;=\; W \;+\; \mathrm{ctx}\cdot m_{kv} \;+\; M_0, \tag{3.1}$$

ahol W a kvantált súlyok mérete, ctx a kontextus hossza tokenben, m_{kv} a KV-gyorsítótár (a már feldolgozott tokenek kulcs- és értékvektorainak tára) tokenenkénti mérete, M_0 pedig a futtatókörnyezet többletigénye. Minden rétegben (L réteg) minden kulcs–érték fejhez (H_{kv} fej) egy kulcs- és egy értékvektort kell tárolni, egyenként d_h elemmel és elemenként 2 bájttal. A Qwen2.5-7B modellre [52] ($L = 28$, $H_{kv} = 4$, $d_h = 128$):

$$m_{kv} = 2 \cdot 28 \cdot 4 \cdot 128 \cdot 2 \text{ bájt} = 57\,344 \text{ bájt} \approx 56 \text{ KiB},$$

így 8 192 tokenes kontextusnál a gyorsítótár kb. 448 MiB, 16 384 tokennél kb. 896 MiB. A 3.3. táblázat több helyi generátorra mutatja a becslést.

A PULI-LlumiX 8B [56] a Llama-3.1-8B architektúráját örökli, így memóriaigénye azonos vele. A Gemma 3 12B-nél az egyszerű képlet túlbecsül: a modell rétegeinek többsége csak

3.3. táblázat. Becsült GPU-memóriaigény 8 192 tokenes kontextusnál, Q4_K_M kvantálással és $M_0 \approx$ $\approx 0{,}6$ GB többlettel. A súlyok a GGUF-fájlok mérete, m_{kv} a publikált architektúrákból számolt érték. A Gemma 3 12B sorában mért érték áll (lásd a szöveget).

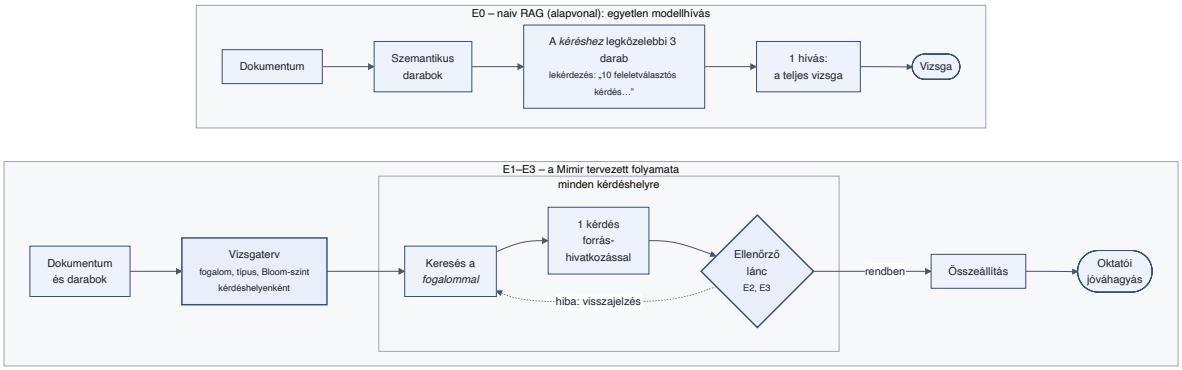
Modell	L/H_{kv}	Súlyok	m_{kv}	Összesen	Kártya
Qwen2.5-3B	36/2	1,9 GB	36 KiB	2,8 GB	8 GB
Qwen2.5-7B	28/4	4,7 GB	56 KiB	5,8 GB	8 GB
Llama-3.1-8B [53] (= PULI-LlumiX 8B)	32/8	4,9 GB	128 KiB	6,6 GB	8 GB
Gemma 3 12B [54]	48/8	8,1 GB	($\leq$ 384 KiB)	$\approx$ 7,8 GiB (mért)	12 GB
Qwen2.5-14B	48/8	9,0 GB	192 KiB	11,2 GB	12 GB

egy rövid, helyi ablakra (*sliding window*) figyel, és csak minden hatodik réteg lát a teljes kontextusra, ezért a gyorsítótár jóval kisebb, mint ctx $\cdot\, m_{kv}$. Ennél a modellnél ezért a mért értéket használjuk: az Ollama 8 192 tokenes kontextusnál kb. 7,8 GiB-ot foglal. A becslés következtetése két fogyasztói szint: a 7–8B modellek 4 bites kvantálással elférnek egy 8 GB-os kártyán, a 12B modell az asztali környezet foglalásával együtt már 12 GB-os kártyát igényel. A mért értékeket az 5. fejezet közli.

A kontextus növelése a kisebb modelleknél sem ingyenes: 32 768 tokennél a Qwen2.5-7B gyorsítótára önmagában 1,75 GiB lenne. Ezért a helyi teljes dokumentumos alapvonal 16 384 tokenes kontextussal és 30 000 karakterre csonkolt dokumentummal fut, a tervezett folyamat minden hívása viszont rövid.

3.5. A generálási módszer

A 3.2. ábra a két összehasonlított generálási folyamatot mutatja. A *naiv RAG* (E0) a felhasználói kéréshez leginkább hasonló három darabot keresi vissza, és egyetlen hívásban kéri a teljes vizsgát; ez a rendszer eredeti működése és a legolcsóbb ésszerű alapvonal. A *tervezett folyamat* (E1–E3) először explicit vizsgatervet készít, majd a terv minden pontjára (a továbbiakban: *kérdéshelyére*) külön keresést, egyetlen kérdés generálását és – az ellenőrző változatokban – ellenőrzést végez.



3.2. ábra. A naiv RAG és a MIMIR tervezett generálási folyamata. A terv a vizsga szerkezetét még az első kérdés megírása előtt rögzíti, és minden kérdést a saját, hivatkozott kontextusával szemben ellenőriz.

A lényegi különbség a feladat felbontása. A naiv alapvonalban a modell egyetlen hívásban dönt a kérdések számáról, témájáról, nehézségéről és disztraktorairól (rossz válaszlehetőségeiről). A tervezett folyamatban a szerkezeti döntéseket determinisztikus program hozza meg, a modell pedig egy-egy rövid kérdést ír, csak a releváns kontextus alapján.

A módszer modellfüggetlen: minden nyelvimodell-hívás egy OpenAI-kompatibilis felületen megy át, a generátor tehát egy konfigurációs kulccsal cserélhető. A vizsgálatokban három helyi generátort használtunk: a Qwen2.5-7B-t [52] (pilot és előzetes vizsgálat), valamint a NYTK magyarra továbbtanított, Llama-3.1 alapú PULI-LlumiX 8B [56] és a Gemma 3 12B [54] modelljét (fő vizsgálat). Mindhárom 4 bites (Q4_K_M) kvantálással fut az Ollamán [59], $T = 0{,}2$ hőmérséklettel és 8 192 tokenes kontextussal (a B-doc-L 16 384 tokennel); a fő vizsgálatban a válasz hossza legfeljebb 4 096 token (`num_predict`).

3.5.1. Szemantikus darabolás

A darabolás első lépése a mondatokra bontás. Ezután minden mondathoz egy 768 dimenziós beágyazóvektort rendelünk a multilingual-E5-base [32] kódolóval. A kódoló hosszabb szöveg-ablakokat dolgoz fel, és egy mondat vektora tokenjei vektorainak átlaga, így a szomszédos mondatok jelentését is tükrözi. Két szomszédos mondat *szemantikus távolsága*

$$\delta_i = 1 - \cos\big(\mathbf{v}_i, \mathbf{v}_{i+1}\big), \tag{3.2}$$

ahol $\mathbf{v}_i$ az i-edik mondat vektora. Témaváltásnál a távolság nagy, ezért ott vágunk, ahol δ_i meghaladja a *vágási küszöböt* (τ). Alapértelmezésben τ a távolságok 85. percentilise, vagyis a legnagyobb 15 % szomszédos távolságnál vágunk. Hogy egy darab ne nőjön túl hosszúra, a küszöb a célméret (1 000 karakter) túllépése után arányosan csökken: egy kétszeres hosszúságú darab már fele akkora távolságnál is lezárul. Ez a *puha korlát* mondat közepén soha nem vág.

A csonkolt kódoló hibája. *A kódoló legfeljebb 512 tokenes bemenetet fogad. Az eredeti implementáció a teljes dokumentumot egy menetben, csonkolva kódolta, így az 512. token után kezdődő mondatok nullvektort kaptak, és hosszabb dokumentumon a vágás már nem a tartalomtól, csak a darab hosszától függött. A javított kódoló mondathatáron vágott, legfeljebb 510 tokenes ablakokban dolgozik (rövid szövegen az eredmény azonos). Az* E5a *konfiguráció a régi kódolót szándékosan megtartja, hogy a hiba hatása mérhető legyen.*

3.5.2. Indexelés és az alapvonalak

A darabokat a rendszer az E5 kódolóval beágyazza, és egy memóriában futó Qdrant [82] vektortárba tölti, amely minden új dokumentum előtt kiürül. A *sűrű keresés* a lekérdezéshez koszinusz-hasonlóság szerint legközelebbi k darabot adja vissza. A *naiv RAG* (E0) lekérdezése a felhasználói kérés szövege („Készíts pontosan 10 darab vizsgakérdést, típus: feleletválasztós…”), kontextusa a három leghasonlóbb darab, promptja pedig a rendszer éles, v1.0-s promptja

(A. függelék). A *teljes dokumentumos prompt* (B-doc-L helyben, B-doc-S a szerveren) ugyanezt a promptot használja, de a kontextus a teljes dokumentum: helyben legfeljebb 30 000, a szerveren 80 000 karakter; a levágott rész arányát rögzítjük.

3.5.3. A vizsgaterv

Fogalomtervezés. A tervező egyetlen hívásban a dokumentum *áttekintését* kapja meg: minden darab azonosítóját ($C_0, C_1, \ldots$) és elejét, összesen legfeljebb 12 000 karakterben (darabonként legalább 120 karaktert). A modell így hosszú, folytonos szöveg helyett azonosítókkal tagolt, rövid felsorolást lát, ami a „lost in the middle" jelenség [6] ellen hat. A tervező $n+3$ fogalmat ad vissza (n a kért kérdésszám), mindegyikhez egy mondatos leírást (mit kell kérdezni), az alátámasztó darabok azonosítóit és egy fontossági értéket (1: részlet, 2: fontos, 3: központi). Értelmezhetetlen válasznál darabonként egy álfogalom lép a helyükre, így a terv mindig létrejön.

Bloom-szintek kiosztása. A kért nehézséghez egy Bloom-szint-eloszlás tartozik [11] (3.4. táblázat), amelyet a *legnagyobb maradék módszerével* (Hamilton-módszer [83]) alakítunk egész darabszámokká: minden szint először a kérdésszám és az arány szorzatának egészrészét kapja, a maradék helyeket pedig a legnagyobb törtrészű szintek. Közepes nehézségnél $n = 7$ kérdésre a szorzatok $(2{,}8;\ 2{,}8;\ 1{,}4)$, az egészrészek $(2,2,1)$, a két maradék hely a két legnagyobb törtrészű szinthez kerül, így a kiosztás $(3,3,1)$. Az összeg mindig pontosan n, így a nehézség nem kérés a promptban, hanem a terv szerkezeti tulajdonsága. A szinteket véletlenszerűen rendeljük a kérdéshelyekhez, a kész vizsgát pedig a könnyebbtől a nehezebb felé rendezzük.

3.4. táblázat. A nehézségi szintekhez rendelt Bloom-eloszlások és egy 10 kérdéses vizsga kiosztása.

Nehézség	Emlékezés	Megértés	Alkalmazás	Elemzés	Értékelés	Kiosztás ($n = 10$)
Könnyű	0,5	0,5	–	–	–	(5, 5, 0, 0, 0)
Közepes	–	0,4	0,4	0,2	–	(0, 4, 4, 2, 0)
Nehéz	–	–	0,3	0,4	0,3	(0, 0, 3, 4, 3)

Lefedettségvezérelt fogalomválasztás. A kérdéshelyekhez egy *mohó* eljárás választ fogalmat: minden lépésben azt, amely még le nem fedett darabot érint, ezen belül a legfontosabbat; ha már minden darab le van fedve, a fontosság dönt. Ha a tervező legalább n fogalmat javasolt, a kérdések így nem tömörülhetnek a dokumentum egyetlen részére. Kevesebb fogalom esetén a fogalmakat más Bloom-szinttel újra felhasználjuk; a ki nem választott fogalmak alkotják a *tartaléklistát*.

3.5.4. Kérdéshelyenkénti keresés

Minden kérdéshely saját kontextust kap. A lekérdezés a fogalom neve és leírása, a kontextus pedig a fogalomhoz a tervező által rendelt első két darab, kiegészítve a keresés $k = 3$ találatával (ismétlés nélkül), legfeljebb 6 000 karakterig. A lényegi különbség a naiv RAG-hoz képest, hogy

a lekérdezés itt egy tartalmi fogalom („immunológiai memória: hogyan alakul ki”), nem pedig feladatleírás („10 kérdés, könnyű”).

A *hibrid* változat (E2h) a sűrű rangsort egy lexikális BM25-rangsorral [30] egyesíti. A magyar ragozás miatt a szavakat egyszerű *előtagtövesítéssel* normalizáljuk: a hat karakternél hosszabb szavakból az első hat karaktert tartjuk meg, így az „immunrendszer”, „immunrendszert”, „immunrendszerben” alakok közös kulcsot kapnak. A két rangsort a reciprok rangfúzió [34] egyesíti; a c darab pontszáma

$$\mathrm{RRF}(c) = \frac{1}{60 + \mathrm{rang}_{\text{sűrű}}(c)} + \frac{1}{60 + \mathrm{rang}_{\mathrm{BM25}}(c)}, \tag{3.3}$$

ahol a rang az 1-től számozott helyezés, a 60-as konstans pedig az irodalomban bevett érték, amely csökkenti az első helyek túlzott súlyát.

3.5.5. Forráshoz kötött kérdésgenerálás

A generátor egyetlen kérdést ír a kérdéshely terve szerint. A prompt (A. függelék) megadja a fogalmat, a kérdezendő tartalmat, a típusszabályt (négy válasz, egy helyes), a Bloom-szint magyar leírását, a nehézséget és a nyelvet; megismétli az éles prompt szabályait; felsorolja a már elfogadott kérdéseket; és megköveteli a `citations` mezőt, amelyben a modell megnevezi a kulcsot igazoló darabok azonosítóit. Így minden kérdés saját, ellenőrizhető forrásrészlethez kötődik (G1).

3.5.6. Az ellenőrző lánc

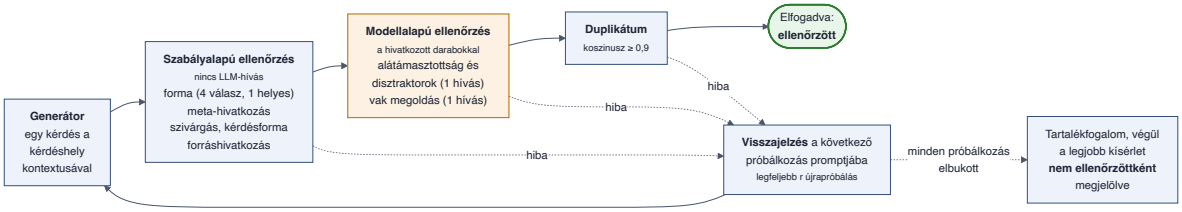
Az ellenőrzések a legolcsóbbtól a legdrágábbig futnak (3.5. táblázat, 3.3. ábra): ha egy szabályalapú ellenőrzés hibát talál, a nyelvi modellt igénylő ellenőrzések el sem indulnak. A modellalapú ellenőrzések forrása a kérdés által hivatkozott darabok.

3.5. táblázat. Az ellenőrző lánc lépései végrehajtási sorrendben, a hiba esetén adott visszajelzéssel.

Ellenőrzés	LLM-hívás	Visszajelzés hiba esetén
Forma	–	feleletválasztós kérdés: 4 válasz, ebből 1 helyes
Meta-hivatkozás	–	fogalmazd meg „a szöveg szerint” jellegű hivatkozás nélkül
Szivárgás és kérdésforma *(a pilot után)*	–	a törzs ne tartalmazza a választ, és legyen valódi kérdés
Hivatkozás	–	hivatkozz a kontextus legalább egy darabjára
Alátámasztottság és disztraktorok	1	a kulcsot a részlet nem igazolja / egy disztraktor igaz lehet / kétértelmű
Vak megoldás	1	egy független megoldó más választ jelölt meg
Duplikátum	–	túl hasonló egy már elfogadott kérdéshez

A *szivárgási szabály* akkor jelez, ha a legalább háromszavas kulcs szavainak legalább 80 %-a a törzsben (a kérdés szövegében) is szerepel, a *kérdésforma-szabály* akkor, ha a törzs sem

kérdőjelre, sem kettőspontra nem végződik; mindkettőt a pilot hibamódja (5.1. szakasz) motiválta. Az *alátámasztottsági* hívás azt kérdezi, igazolja-e a forrás a kulcsot, hamis-e minden disztraktor és kétértelmű-e a kérdés; a *vak megoldásban* a modell a kulcs ismerete nélkül, összekevert válaszokkal dönt. *Duplikátum* a legalább 0,9 koszinusz-hasonlóságú kérdés. Az ellenőrzést maga a generátor végzi, az E2x-ben egy másik modell (Llama-3.1-8B [53]).



3.3. ábra. Az ellenőrző lánc visszacsatolással. Az olcsó szabályalapú ellenőrzések előbb futnak; a hiba szövege a következő generálási kísérlet promptjába kerül.

Egy kérdéshely kitöltését a 3.1. algoritmus írja le. A hibák listája visszajelzésként („Az előző próbálkozás hibás volt, javítsd: …”) a következő kísérlet promptjába kerül; ha a kulcs nem igazolt, a keresés két darabbal bővül. Ha egy tartalékfogalommal sem sikerül, a *legjobb* kísérlet (formailag helyes, szivárgásmentes, legkevesebb hibájú) marad meg *nem ellenőrzöttként* megjelölve, hogy az oktató lássa; hibás formánál egy *javító* hívás pótolja a hiányzó disztraktorokat. Az E1 csak a formát, a meta-hivatkozást és a duplikátumot vizsgálja. Technikai hiba miatt elmaradt ellenőrzés a kérdést nem tartja fel, de naplózzuk.

Algoritmus 3.1: Egy kérdéshely kitöltése visszacsatolással (E2).

Bemenet: kérdéshely s (fogalom, típus, Bloom-szint); elfogadott kérdések $\mathscr{A}$; $k = 3$; $r = 1$
Kimenet: kérdés q és az *ellenőrzött* jelző

$q^* \leftarrow \emptyset$;
minden *fogalom* $c \in (s.\text{fogalom},\ \textit{az első tartalékfogalom})$ **elemre**
  $h \leftarrow \emptyset$; $k' \leftarrow k$;
  ciklus $t = 0, \ldots, r$ **:**
    $K \leftarrow \textsc{Kontextus}(c, k')$; // $\leq$ 6000 karakter
    $q \leftarrow \textsc{Generál}(s, c, K, \mathscr{A}, h)$; // h: az előző hibák
    $h \leftarrow \textsc{Ellenőriz}(q, K) \cup \textsc{Duplikátum}(q, \mathscr{A})$;
    ha $h = \emptyset$ **akkor visszatér** $(q,$ igaz$)$;
    $q^* \leftarrow$ a jobb q^* és q közül;
    ha *a kulcs nem igazolt* **akkor** $k' \leftarrow k' + 2$;
ha q^* *formája hibás* **akkor** $q^* \leftarrow \textsc{Javít}(q^*)$;
visszatér $(q^*,$ hamis$)$;

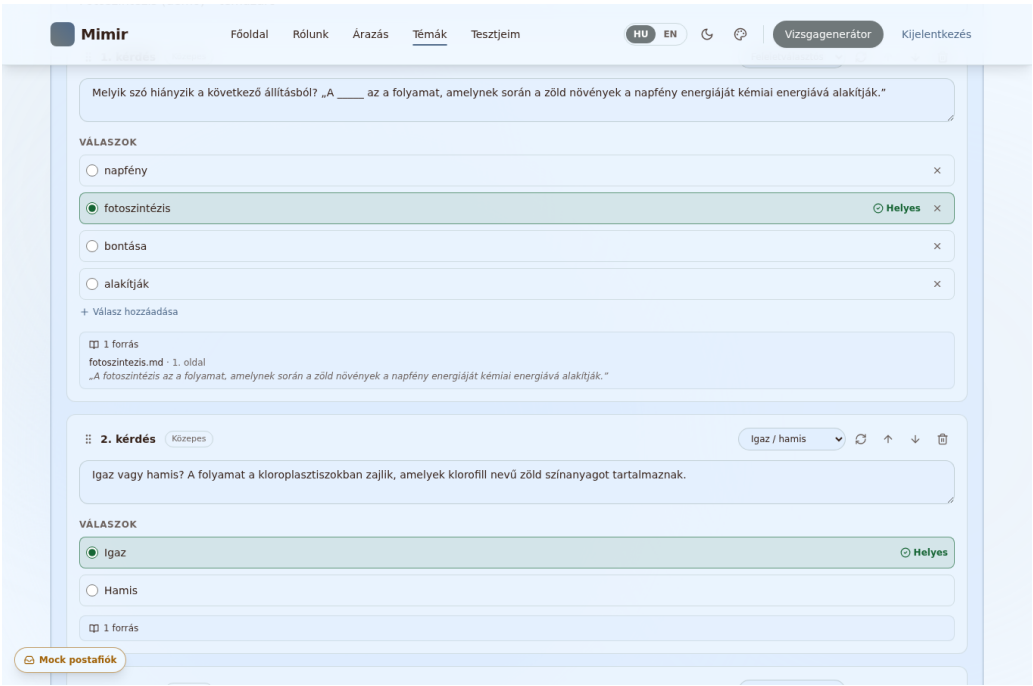
Költség. A modellhívások száma a módszer fő költségtényezője. Az egyhívásos konfigurációk (E0, E5a–E5c, B-doc) egy hívással dolgoznak. Az ellenőrzés nélküli E1 legalább $1 + n$ hívást igényel (egy tervezés és kérdésenként egy generálás), az ellenőrzött E2 legalább $1 + 3n$ hívást (kérdésenként generálás, alátámasztottsági ellenőrzés és vak megoldás). Tíz kérdésnél ez 11, illetve 31 hívás, ha minden kérdés elsőre átmegy; minden elutasítás további két-három hívással jár.

3.5.7. Munkamenet-hatókörű fogalomgráf

Az E3 konfiguráció a tervezőhívást egy fogalomgráffal váltja ki. A darabokat legfeljebb 3 000 karakteres kötegekbe csoportosítjuk, és kötegenként egy hívásban kinyerjük a fogalmakat (definícióval, szülőfogalommal, darabazonosítókkal), a rövid tényeket és a fogalmak közötti kapcsolatokat (*fajtája*, *része*, *okozza*, *ellentéte* stb.); a nagyon hasonló nevű fogalmakat (névbeágyazásuk koszinusz-hasonlósága legalább 0,92) összevonjuk, ami a ragozott alakokat is egyesíti. A fogalmakat a $\text{cent}(v) = \text{kapcsolatok száma} + \frac{1}{2} \cdot \text{darabok száma} + \frac{1}{2} \cdot \text{tények száma}$ centralitás szerint rendezzük, és ez a lista lép a tervező helyébe. A generátor disztraktor-ötletként a fogalom *testvérfogalmait* kapja (azonos szülőjű fogalmakat, például a „T-sejt” mellett a „B-sejtet”), a kontextus pedig a szomszédos fogalmak darabjaival bővül. A gráf csak a vizsga élettartamáig létezik, a memóriában.

3.5.8. Összeállítás és emberi felügyelet

Az elfogadott kérdéseket Bloom-szint szerint rendezzük, és mindegyikhez rögzítjük a hivatkozott darabokat, a tervezett fogalmat és az *ellenőrzött* jelzőt. A vázlat a webes szerkesztőbe kerül (3.4. ábra), ahol az oktató szerkesztheti, átrendezheti és törölheti a kérdéseket, módosíthatja a kulcsot, egyetlen kérdést újragenerálhat, és minden kérdés alatt látja a hivatkozott forrásrészletet a fájl nevével és oldalszámával. Jóváhagyás nélkül semmi sem exportálható, az exportált dokumentum pedig feltünteti, hogy a tartalom MI-rendszer közreműködésével, melyik modellel készült (G5).



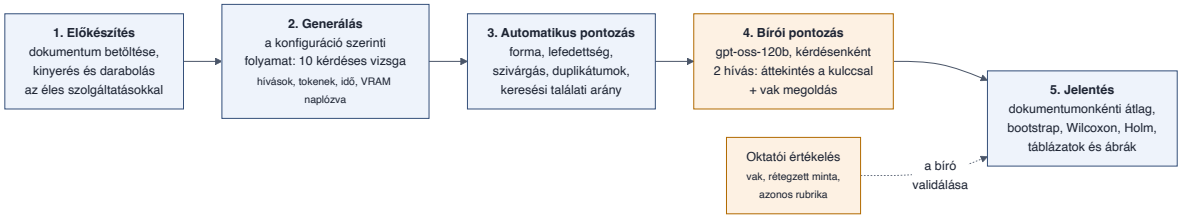
3.4. ábra. A webes szerkesztő egy generált feleletválasztós kérdéssel: a megjelölt helyes válasz, alatta a hivatkozott forrásrészlet („1 forrás”, fájlnév és oldalszám); jobb felül a típusválasztó, az újragenerálás, az áthelyezés és a törlés gombja.

4. Kiértékelési módszertan

A 2. fejezetben láttuk, hogy a kérdésgenerálás kiértékelése többnyire referenciakérdésekkel való szóegyezésre és kis, nem vak emberi értékelésre támaszkodik, és ritkán ellenőrzi a kulcs helyességét. Az itt bemutatott módszertan ezzel szemben (i) a forráshoz viszonyított helyességet méri, (ii) egytényezős kísérleti tervvel választja szét a komponensek hatását, (iii) a mérőeszközt, vagyis a kérdéseket pontozó, független nyelvi modellt (a *bírót*) is ellenőrzi, és (iv) a dokumentumot tekinti a statisztikai elemzés egységének. Minden eleme a nyílt forráskódú keretrendszerben (`tests/eval`) implementált.

4.1. A tesztelés menete

A kiértékelés alapegysége egy *teszt*: egy *konfiguráció* (a generálási folyamat egy rögzített beállítása, rövid jele például E0 vagy E2) egy dokumentumon, egy véletlenmaggal egy tízkérdéses feleletválasztós vizsgát állít elő, amelyet ezután automatikus mutatókkal és a bíróval pontozunk (4.1. ábra). Az előkészítés (szövegkinyerés, darabolás, indexelés) a futó szolgáltatásokon át, az éles rendszerrel azonos módon történik. Generálás közben minden modellhívás szakaszát, tokenszámát és idejét rögzítjük, a GPU-memóriát egy háttérszál mintavételezi.



4.1. ábra. Egy teszt lépései: előkészítés a szolgáltatásokon, generálás a konfiguráció szerint, automatikus mutatók, bírói pontozás és összesítés.

A reprodukálhatóságot négy mechanizmus biztosítja: (1) minden futás rögzíti a teljes konfigurációt, a kód és az adatkészlet verzióját és a hardvert; (2) a bíró válaszai gyorsítótárba kerülnek, így az újrapontozás determinisztikus; (3) a megszakított futások folytathatók; (4) a keretrendszert 43 automatikus teszt fedi le, köztük ál-szolgáltatásokkal és ál-nyelvi modellel a teljes folyamat.

4.2. Adatkészlet

Az `eval-v1` adatkészlet 39 dokumentumból áll, három forrásból (4.1. táblázat). Minden referenciakérdéshez rendelkezésre áll a forrásszakasz *és* a választ igazoló mondat, mert a keresési mutatók ezt igénylik. A három saját, rövid magyar oktatási szöveghez (kávé, immunrendszer, játékelmélet) négy-négy kézzel írt referenciakérdés tartozik; az EduQG [21] OpenStax-tankönyvrészletekhez írt szakértői kérdéseket, a MILQA [80] magyar Wikipédia-cikkekre épülő

szövegértési kérdéseket ad. Az importált forrásokból csak azokat a kérdéseket tartottuk meg, amelyek igazoló mondata szó szerint szerepel a dokumentumban.

4.1. táblázat. Az `eval-v1` adatkészlet és a fő vizsgálatban használt része. R/K/H: rövid ($\leq 6\,000$), közepes (6 000–25 000) és hosszú (25 000–90 000 karakteres) dokumentumok.

		`eval-v1`			Fő vizsgálat		
Forrás	Nyelv	Dok.	R/K/H	Ref. kérdés	Dok.	R/K/H	Licenc
Saját oktatási szövegek	magyar	3	3/0/0	12	3	3/0/0	saját
OpenStax, EduQG	angol	12	4/4/4	74	3	1/1/1	CC BY 4.0
Wikipédia, MILQA	magyar	24	8/8/8	240	24	8/8/8	CC BY-SA 4.0
Összesen		39	15/12/12	326	30	12/9/9	

A fő vizsgálat a 39 dokumentumból 30-at használt: mind a 24 MILQA-cikket, a 3 saját szöveget és 3 OpenStax-részletet (`en-economics-short-01`, `en-humanities-medium-02`, `en-law-long-03`), összesen 27 magyar és 3 angol dokumentumot. A szűkítés oka a futásidő volt; az elsődleges cél a magyar nyelvű kiértékelés, az angol részletek a nyelvi összevetést szolgálják.

A referenciakérdések *nem* célok, hiszen egy jó vizsgakérdésnek nem kell hozzájuk hasonlítania; a keresés pontosságát és a tematikus egyezést mérjük velük (4.5. szakasz).

4.3. A kísérleti konfigurációk

A kísérleti terv *egytényezős*: minden konfiguráció egy referenciakonfigurációhoz képest pontosan egy tényezőben tér el, így a kettő közötti különbség az adott tényezőnek tulajdonítható (4.2. ábra). A 4.2. táblázat a 14 konfigurációt három csoportban foglalja össze.

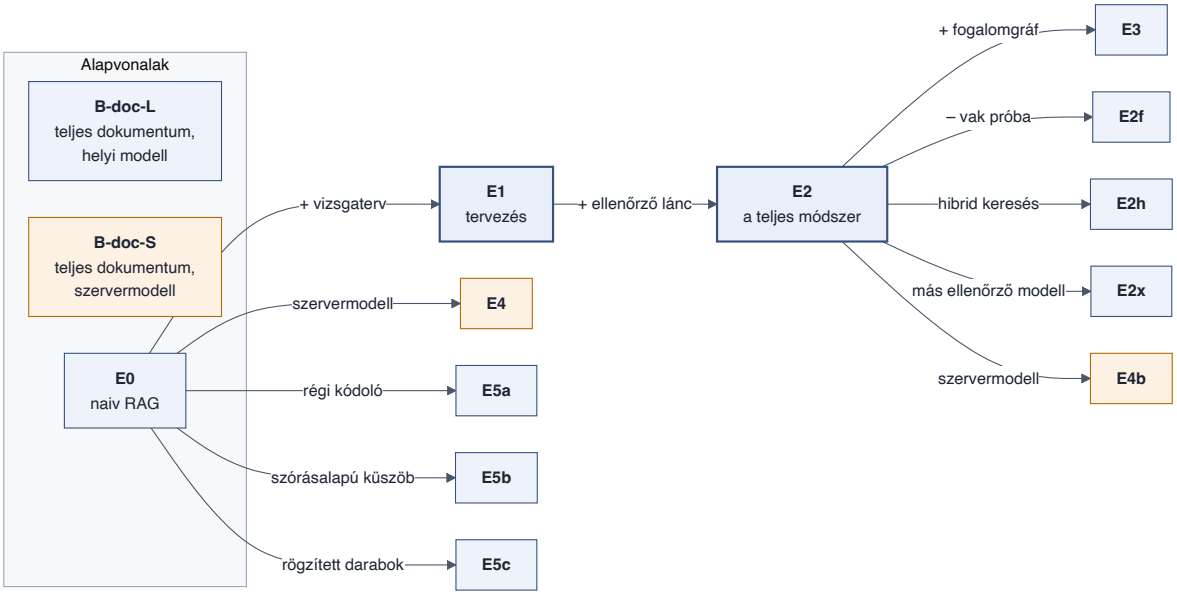
Alapvonalak. Azt modellezik, amit egy oktató a MIMIR nélkül tenne. A B-doc-S a mai gyakorlat: a teljes dokumentum és az éles prompt egy hívásban kerül a nagy szervermodellhez, mintha az oktató a jegyzetét egy csevegő asszisztensbe töltené fel. A B-doc-L ugyanez a helyi modellel, 16 384 tokenes kontextussal; azt mutatja, mit ad egy kis modell, ha a feladatot nem bontjuk fel. Az E0 a legegyszerűbb visszakereséses megoldás és a rendszer eredeti működése.

A módszer lépései. Ezek a konfigurációk a MIMIR komponenseit egyenként kapcsolják be. Az E1–E0 különbség a tervezés hatása (1. és 3. hipotézis), az E2–E1 az ellenőrzésé (2. hipotézis), az E3–E2 a fogalomgráfé. Az E2-ben az ellenőrzést ugyanaz a modell végzi, amely a kérdést írta.

Kiegészítő ablációk. Az E2f a lánc legdrágább elemét, a vak megoldást hagyja el; az E2h azt vizsgálja, segít-e a lexikális keresés; az E2x azt, hogy jobb-e egy a generátortól eltérő ellenőrző (a PULI-LlumiX esetében ez a generátor alapmodellje, tehát nem független család). Az E4 és az E4b a helyi modellt a nagy szervermodellre cseréli (4. hipotézis), az E5a–E5c pedig az E0 darabolását változtatja meg: a régi, hibás kódoló (3.5.1. szakasz), egy másik küszöbszabály és a tartalomtól független, rögzített darabolás [36].

4.2. táblázat. A kísérleti konfigurációk. Ref.: amelytől a konfiguráció egyetlen tényezőben eltér; Magok: dokumentumonkénti ismétlések száma az előzetes és a fő vizsgálatban (a pilotban mindenhol 3); Vizsg.: P = pilot, Ev = előzetes vizsgálat, F = fő vizsgálat.

Jel	Mit változtat	Ref.	Magok	Vizsg.
Alapvonalak				
B-doc-L	teljes dokumentum egy promptban, helyi modell ($\leq 30\,000$ karakter)	–	3	Ev, F
B-doc-S	ugyanez a szervermodellel ($\leq 80\,000$ karakter)	–	3	Ev
E0	naiv RAG: a kéréshez legközelebbi 3 darab, a vizsga egy hívásban	–	3	P, Ev, F
A módszer lépései				
E1	vizsgaterv, kérdéshelyenkénti keresés és generálás, ellenőrzés nélkül	E0	1	P, Ev, F
E2	E1 + ellenőrző lánc visszacsatolással (a teljes módszer)	E1	1	P, Ev, F
E3	E2 + fogalomgráf	E2	1	P, Ev, F
Kiegészítő ablációk				
E2f	E2 vak megoldási próba nélkül	E2	1	P, Ev, F
E2h	E2 hibrid (BM25 + sűrű) kereséssel	E2	1	P, Ev, F
E2x	E2, az ellenőrzést egy másik modell (Llama-3.1-8B) végzi	E2	1	Ev, F
E4	E0 a szervermodellel	E0	3	P, Ev
E4b	E2 a szervermodellel	E2	1	Ev
E5a	E0 a régi, 512 tokenre csonkolt kódolóval	E0	3	P, Ev, F
E5b	E0, vágási küszöb: átlag $+1{,}5$ szórás	E0	3	P, Ev, F
E5c	E0 rögzített, 800 karakteres darabokkal	E0	3	P, Ev, F



4.2. ábra. A konfigurációk egytényezős fája: minden él egyetlen tényező megváltoztatása.

A tervezett konfigurációk egy maggal futnak, mert egy vizsga perceket igényel, és a variancia fő forrása a dokumentum; az olcsó egyhívásos konfigurációk zaját három mag csökkenti. A tervben szereplő E0s kontrollkonfiguráció (az E0 az éles végponton át) egyik vizsgálatban sem futott. Előre rögzített, a többszörös összehasonlítás korrekciójába tartozó páros összehasonlítás hat van: E1–E0 (tervezés), E2–E1 (ellenőrzés), E2–E0 (teljes módszer), E2–B-doc-L (módszer vs. teljes dokumentumos prompt), E2x–E2 (ellenőrző modell) és E3–E2 (gráf). A többi összevetés leíró jellegű.

4.4. A három vizsgálat

A kísérleteket három lépcsőben futtattuk (4.3. táblázat). A *pilot* a mérési láncot próbálta ki; egy dokumentumon hipotézist eldönteni elvileg sem lehet (4.8. szakasz). Az *előzetes vizsgálat* a pilot után javított folyamattal (szivárgási szabályok, javító hívás, $r = 1$, a bíró a kérdés forrásdarabjait kapja), a *fő vizsgálat* ugyanezzel, 30 dokumentumon és két helyi generátorral futott.

4.3. táblázat. A három vizsgálat beállításai.

	Pilot	Előzetes vizsgálat	Fő vizsgálat
Időpont (2026)	szept. 23.	szept. 27–28.	okt. 7. – nov. 1.
Dokumentumok	1 (`hu-coffee`, 1 702 karakter)	9 (3 saját magyar, 6 angol); konfigurációnként 6	30 (24 MILQA, 3 saját, 3 OpenStax)
Konfigurációk	10	14	11 helyi, mindkét generátorral
Generátor	Qwen2.5-7B (E4: szervermodell)	Qwen2.5-7B; szervermodell (B-doc-S, E4, E4b)	PULI-LlumiX 8B és Gemma 3 12B
Ismétlés	3 mag mindenhol	3 mag (egyhívásos konfigurációk), 1 mag (tervezett konfigurációk)	mint az előzetes vizsgálatban
Bírált kérdések	300	1 670	PULI: 2 935; Gemma: 6 212
Újrapróbálkozás (r)	2	1	1
Szivárgási szabály, javító hívás	nincs	van	van
Bíró forrása	a teljes dokumentum	a kérdés forrásdarabjai (teljes dokumentumos konfigurációknál a dokumentum első 24 000 karaktere)	mint az előzetes vizsgálatban
GPU	RTX 3070 Ti (8 GB), asztali PC	RTX A4500 (20 GB), egyetemi munkaállomás	RTX A4500 (20 GB), egyetemi munkaállomás

Az előzetes vizsgálatban az E4 és az E4b hat angol dokumentumon futott (ebből három közös), a többi konfiguráció a közös hat dokumentumon. A PULI-LlumiX-nél azért kevesebb a bírált kérdés, mert gyakran tíznél kevesebb kérdést adott vissza. A három vizsgálat kissé eltérő

implementációt mér (a pilot az akkori, a másik kettő a javított folyamatot, más dokumentumokon és generátorral), ezért összehasonlítást mindig egy vizsgálaton belül végzünk. Az előzetes és a fő vizsgálat egy 20 GB-os munkaállomás-kártyán futott; a modell, a kvantálás és a kontextus ugyanaz, mint egy fogyasztói kártyán, így ez a minőséget nem befolyásolja, a memóriamérés a modell saját foglalását adja, a futásidők viszont csak azonos hardveren vethetők össze.

4.5. Mutatók

Minden ráta 0 és 1 (0 és 100 %) közötti, és a szivárgás, a nem kérdő törzs és a duplikátumok kivételével a nagyobb érték a jobb.

Tartalmi mutatók. Az *alátámasztottság* (G) azon kérdések aránya, amelyeknél a bíró szerint a forrás igazolja a megjelölt kulcsot; a bírónak az igazoló mondatot szó szerint idéznie kell. A *vak megoldhatóság* (A) azoké, amelyeket a bíró a kulcs ismerete nélkül, összekevert válaszokkal, csak a forrás alapján helyesen old meg; ez a kulcs helyességét és a kérdés egyértelműségét együtt méri. A *disztraktor-validitás* (V) a kérdésenkénti azon disztraktorok aránya, amelyek a forrás szerint hamisak *és* témába vágók. A *formai megfelelés* (F) vizsgaszintű igen/nem érték: a vizsga pontosan a kért számú kérdést tartalmazza, és minden kérdésnek pontosan négy válasza és egy kulcsa van.

Minőségi index. A négy mutatót egy számmá foglaljuk össze:

$$Q = \frac{F + G + A + V}{4}. \tag{4.1}$$

A Q szándékosan egyszerű, súlyozatlan átlag, amelyet vizsgánként számolunk; a sikertelen vizsga (értelmezhetetlen modellválasz) Q-ja 0, a technikai (szolgáltatás-) hibák kimaradnak. Egyetlen hibás formájú vagy hiányzó kérdés így a Q-t 0,25-tel csökkenti; ezt vállaljuk, mert egy hiányos vagy hibás kulcsú vizsga nem használható változtatás nélkül. A komponenseket külön is közöljük.

Rubrika. A bíró emellett négy szempontot 1–5 skálán pontoz – *helyesség*, *érthetőség*, *disztraktorok*, *szintillesztés* –, és egy igen/nem ítéletet ad (*felhasználnám*: bekerülne-e egy valódi dolgozatba legfeljebb apró javítással). A rubrika szövege (B. függelék) a bíró és az oktatók számára azonos, ami a bíró validálásának (5. hipotézis) előfeltétele.

Lefedettség. Azt méri, mennyire oszlik el a vizsga a tananyagon (a P2 probléma mérőszáma). Minden kérdéshez (törzs és kulcs együtt) megkeressük a beágyazás szerint legközelebbi darabot:

$$\mathrm{Lef} = \frac{\text{a kérdésekhez legközelebbi különböző darabok száma}}{K}, \tag{4.2}$$

ahol K a dokumentum darabjainak száma. Egy tízkérdéses vizsga lefedettsége legfeljebb $10/K$, így egy százdarabos dokumentumon legfeljebb 10 %; a mutató ezért csak azonos dokumentumon,

konfigurációk között hasonlítható, és szerkezeti, nem minőségi mutatóként a Q-ba nem számít bele.

Szivárgás és nem kérdő törzs. Ugyanazzal a szabállyal mérjük, mint az ellenőrző (3.5.6. szakasz): *szivárgás*, ha a legalább háromszavas kulcs szavainak legalább 80 %-a a törzsben is szerepel (egy egyszavas tulajdonnév így nem ad téves jelzést); *nem kérdés* a törzs, ha sem kérdőjelre, sem kettőspontra nem végződik (vö. 5.1. szakasz).

További mutatók. A *duplikátumráta* azon kérdéspárok aránya, amelyek koszinusz-hasonlósága meghaladja a 0,9-et; a *meta-hivatkozás* aránya azoké, amelyek „a szöveg szerint” jellegű fordulatot tartalmaznak. A *keresési találati arány* (recall@3) azt méri, hogy a referenciakérdésekkel futtatott keresés első három találata között szerepel-e az igazoló mondatot tartalmazó darab; a *tematikus hasonlóság* azt, hogy a referenciakérdések mekkora részéhez van beágyazás szerint közeli generált kérdés. *Költségmutató* a futási idő (szakaszonként), a modellhívások és tokenek száma, valamint a GPU-memória alapszintje és csúcsa.

4.6. A bíró

A bíró a gpt-oss-120b [66] modell az egyetemi GenAI-szerveren, nulla hőmérséklettel és alacsony („low”) gondolkodási szinttel. Más modellcsaládba tartozik, mint bármelyik generátor (Qwen, Llama/PULI, Gemma), így az önpreferencia torzítása [65] nem lép fel. Kérdésenként két hívást végez: az *áttekintő* hívás látja a kulcsot (alátámasztottság szó szerinti idézettel, disztraktoronkénti ítélet, rubrikapontok, megvalósult Bloom-szint, „felhasználnám”), a *vak* hívás nem, és permutált válaszokkal kell megoldania a kérdést.

Forrásként a bíró a kérdéshez tartozó darabokat kapja (a tervezett konfigurációknál a kérdéshely kontextusát, az `E0`-nál a visszakeresett darabokat), nem a teljes dokumentumot. Ezt a megosztott szerver futásideje indokolta; ára, hogy a dokumentum más részeiben lévő ellentmondást a bíró nem látja. A teljes dokumentumos konfigurációknál a bíró a dokumentum első 24 000 karakterét kapja, így egy hosszú dokumentum későbbi részéből írt kérdést nem tud igazolni; ezeknél az alátámasztottság alsó becslés.

4.7. Oktatói értékelés

A bíró csak akkor érvényes mérőeszköz, ha ítéletei az emberi szakértőkével egyeznek (5. hipotézis). Az oktatói értékelés a dolgozat lezárásakor folyamatban van; protokollját a keretrendszer `rate-export` és `rate-import` lépése valósítja meg. Alapbeállításban a kiválasztott konfigurációkból konfigurációnként 30 kérdést mintavételezünk (előbb minden dokumentumból egyet), 3 értékelővel és 10 közös kalibrációs kérdéssel. Az xlsx-lapok vakok: a konfiguráció neve nem szerepel, a sorrend értékelőnként eltérő véletlen sorrend; minden kérdés mellett egy, minden konfigurációnál azonos módon kiválasztott forrásrészlet áll, a rubrika a bíróéval azonos. Az

oktatók egymás közötti egyezését szempontonként Krippendorff-α-val [68], a bíró és az oktatók egyezését Spearman-féle rangkorrelációval és Krippendorff-α-val, a bináris ítéletekre (a bíró „alátámasztott” ítélete vs. oktatói helyesség ≥ 4) Cohen-κ-val mérjük. Az eredményeket az 5.10. szakasz közli.

4.8. Statisztikai elemzés

Az elemzés egysége a dokumentum. Egy vizsga tíz kérdése nem független: ugyanabból a dokumentumból, egy futáson belül készül, így független megfigyelésként kezelve a próbák a valósnál biztosabbnak mutatnák az eredményt. Ezért minden mutatót előbb dokumentumon belül átlagolunk (a kérdésekre és a magokra), és a következtetés egysége a *dokumentum*; a konfigurációnkénti érték a dokumentumonkénti átlagok átlaga. Bizonytalanságát *bootstrap*-módszerrel [84] becsüljük: a dokumentumokból 10 000-szer húzunk visszatevéssel, és a kapott átlagok középső 95 %-a adja a 95 %-os konfidenciaintervallumot.

Páros összehasonlítás. Két konfigurációt a páros Wilcoxon-féle előjeles rangpróbával [85] vetünk össze, amely a dokumentumonkénti különbségek rangjaival dolgozik, és nem feltételez normális eloszlást (ez a plafonhoz közeli rátáknál lényeges). A hatás nagysága a rangbiszeriális korreláció [86]:

$$r_{rb} = \frac{W^+ - W^-}{W^+ + W^-}, \tag{4.3}$$

ahol W^+ és W^- a pozitív, illetve a negatív különbségek rangösszege; $+1$, ha az új konfiguráció minden dokumentumon jobb, -1, ha mindenütt rosszabb.

A minimális mintaméret. D dokumentum esetén a legkisebb elérhető kétoldali p-érték $2 \cdot 2^{-D}$, amelyet akkor kapunk, ha minden különbség azonos előjelű. Hat dokumentumnál ez 0,031, ötnél 0,0625: a pilot ezért csak leíró, az előzetes vizsgálat hat dokumentuma pedig a legkisebb szám, amelyen egy összevetés egyáltalán szignifikáns lehet. A fő vizsgálat 30 dokumentumán a szignifikancia valóban elérhető.

Többszörös összehasonlítás. A hat előre rögzített összehasonlítás (4.3. szakasz) p-értékeit Holm módszerével [72] korrigáljuk: növekvő sorrendben a legkisebbet 6-tal, a következőt 5-tel stb. szorozzuk, és egy összevetés csak akkor szignifikáns, ha korrigált értéke (és minden nála kisebbé) 0,05 alatt van. Hat dokumentumnál a legkisebb elérhető $p = 0{,}031$ korrigált értéke 0,19, vagyis az előzetes vizsgálatban egyetlen összevetés sem lehet korrekció után szignifikáns.

Egyezés és az ellenőrző mint osztályozó. Két igen/nem ítéletsor (például az ellenőrzőé és a bíróé) egyezését Cohen-κ-val [67] mérjük:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \tag{4.4}$$

ahol p_o a megfigyelt, p_e a véletlenül várható egyezési arány; $\kappa = 0$ a véletlen szintje, $\kappa = 1$ a teljes egyezés, az értelmezés Landis és Koch [69] skáláját követi. Az ellenőrző lánc minden kérdést *átengedett* vagy *megjelölt* kategóriába sorol; a bíró alátámasztottsági ítéletét referenciának tekintve

$$\text{pontosság} = \frac{\text{átengedett és alátámasztott}}{\text{átengedett}}, \qquad \text{felidézés} = \frac{\text{átengedett és alátámasztott}}{\text{alátámasztott}}. \quad (4.5)$$

Hasznos ellenőrzőtől mindkettő magas értéke és $\kappa \gg 0$ várható; $\kappa \approx 0$ esetén a jelzése az oktatónak nem hordoz információt.

5. Eredmények

Ebben a fejezetben a három vizsgálat (4.4. szakasz) eredményeit közöljük. A pilot és az előzetes vizsgálat eredményeit röviden, a módszer fejlesztését meghatározó tanulságokra összpontosítva mutatjuk be (5.1. és 5.2. szakasz); a fejezet nagyobb része a 30 dokumentumos fő vizsgálattal foglalkozik (5.3–5.9. szakasz). Minden közölt szám a nyers kimenetekből, a `tdk/elemzes/elemzes.py` szkripttel készült (C. függelék). Az eredmények értelmezése és a hipotézisek értékelése a 6. fejezetben következik.

5.1. A pilot: három tanulság

A pilot egyetlen rövid magyar szövegen (`hu-coffee`, 1 702 karakter, négy darab) futott tíz konfigurációval, konfigurációnként három vizsgával, összesen 300 kérdéssel, a Qwen2.5-7B modellel. Egyetlen dokumentumon semmilyen különbség nem szignifikáns (4.8. szakasz), a pilot célja ezért a mérési lánc kipróbálása volt. Három tanulsága közvetlenül alakította a módszert.

1. Csendes tartalékágak. Az ellenőrző konfigurációk mindegyikében előfordult, hogy egy kérdéshely minden kísérlete elbukott, és a megtartott végső kérdésnek csak két-három választási lehetősége volt; ez a vizsga formai megfelelését nullára, a minőségi indexét 0,25-tel csökkentette. Hasonló, a mérést *felfelé* torzító hibát a keretrendszer építése során is találtunk: ha minden modellhívás sikertelen volt, a háttérrendszer egy beégetett, egykérdéses „tartalékvizsgát” adott vissza hibaüzenet helyett, a darabolás kódolója pedig a hosszabb dokumentumok nagy részét a tartalomtól függetlenül vágta (3.5.1. szakasz). Mindhármat javítottuk: a végső kérdés kiválasztása a formát is figyeli, egy javító hívás pótolja a hiányzó disztraktorokat, a hibás kódoló hatását pedig az E5a konfiguráció méri.

2. Válaszszivárgás: egy új hibamód. A kérdések kézi átolvasása olyan hibát tárt fel, amelyet sem az automatikus mutatók, sem a bíró nem büntetett: egyes ellenőrzött „kérdések” egy forrásmondatot másoltak a törzsbe, és ugyanezt a mondatot ismételték meg kulcsként. Az E3 egyik kérdése például szó szerint ez volt: „A kávéfogyasztás szokása Etiópiából hamar átterjedt a szomszédos Jemenbe, majd az arab világba, Európába pedig a 17. században jutott el a velencei kereskedők révén.” A kulcs ugyanez a mondat volt, a disztraktorok pedig ugyanez a 16., 18., illetve 15. századdal. A szivárgás az ellenőrző konfigurációkban fordult elő, és ott volt a legerősebb, ahol a legtöbb újrapróbálkozás történt: az E3 kérdéseinek 20 %-a szivárgott, 47 %-uk törzse nem is kérdés volt. A bíró ezeknek többnyire 5-ös helyességi pontot adott, hiszen a kulcs valóban a forrásból származik, és a kérdés vakon is megoldható. Értelmezésünk szerint az ellenőrző visszacsatolása a kis modellt a forrásszöveg szó szerinti átvétele felé tolta, mert ez a legolcsóbb módja annak, hogy egy alátámasztottsági ellenőrzésen átjusson – a Goodhart-törvény [51] tankönyvi példája. Azóta a szivárgást és a nem kérdő törzset az ellenőrző lánc szabályként elutasítja, és mindkettőt mutatóként is mérjük.

3. A futásról futásra jelentkező zaj. Az E0 és az E5a a pilotban azonos darabokat és azonos visszakeresett kontextust kapott (512 tokennél rövidebb szövegen a régi és az új kódoló ugyanazt adja), mégis 0,83, illetve 0,90 minőségi indexet ért el. A GPU-n végzett következtetés tehát alacsony hőmérsékleten és rögzített maggal sem bitre reprodukálható, és egyetlen dokumentumon a 0,07-nél kisebb különbségek zajnak tekintendők. A zaj csökkentésének hatékony módja nem a magok, hanem a dokumentumok számának növelése – ezért futott a fő vizsgálat 30 dokumentumon.

5.2. Az előzetes vizsgálat: Qwen2.5-7B és a szervermodell

Az előzetes vizsgálat a javított rendszerrel, a Qwen2.5-7B modellel futott hat dokumentumon (a három saját magyar szövegen és három angol OpenStax-szövegen), mind a 14 konfigurációval. Ez az egyetlen vizsgálat, amelyben a nagy szervermodell is szerepel, így a helyi és a szerveroldali megoldás közvetlen összevetésére ez ad lehetőséget (5.1. táblázat).

5.1. táblázat. Az előzetes vizsgálat fő eredményei (Qwen2.5-7B, hat dokumentum; az egyhívásos konfigurációk három, a tervezettek egy maggal). Q: minőségi index 95 %-os bootstrap-intervallummal; F, G, A, V: formai megfelelés, alátámasztottság, vak megoldhatóság, disztraktor-validitás; Lef.: lefedettség (%). *Az E4 és az E4b hat angol dokumentumon futott, amelyek közül három közös a többiekével.

Konfiguráció	Q	95 % CI	F	G	A	V	Lef.	Perc	Hívás
B-doc-L (teljes dok., helyi)	0,83	0,76–0,90	83	79	86	85	45	0,4	1
B-doc-S (teljes dok., szerver)	0,95	0,87–0,99	100	90	100	89	47	4,0	1
E0 (naiv RAG)	0,64	0,46–0,81	100	43	62	52	25	0,3	1
E1 (tervezés)	0,88	0,82–0,94	100	80	80	91	46	1,9	18
E2 (tervezés + ellenőrzés)	**0,96**	0,93–0,98	100	90	97	96	44	3,5	51
E4* (naiv RAG, szerver)	0,91	0,80–0,98	100	77	98	87	9	5,3	1
E4b* (E2, szerver)	0,99	0,97–1,00	100	98	100	98	28	47,7	37

Négy megfigyelés emelhető ki. (1) Ugyanazzal a 7B modellel a tervezett és ellenőrzött folyamat (E2, 0,96) jóval jobb, mint a naiv RAG (0,64) és a teljes dokumentumos prompt (0,83), és eléri a teljes dokumentumot kapó szervermodellt (0,95). Az E2 mind a hat dokumentumon legalább 0,89-et ért el. (2) Az ellenőrzés az E1-hez képest öt dokumentumon javított, egyen sem rontott (+0,08). (3) A naiv RAG a három angol dokumentumon összeomlott (0,40; a magyar szövegeken 0,88) – ennek okát a fő vizsgálat kapcsán az 5.5. szakasz elemzi. (4) A három közös angol dokumentumon a helyi E2 (0,95) a szervermodell naiv RAG-változatát (0,93) is elérte; ugyanaz a folyamat a szervermodellel (E4b) közel hibátlan vizsgát adott (0,99), ami a módszer modellfüggetlenségét mutatja.

Az előzetes vizsgálat egy bírói vakfoltot is feltárt: a teljes dokumentumos konfigurációknál a bíró a dokumentumnak csak az első 24 000 karakterét látja, ezért a 68 000 karakteres jogi szövegen a szervermodell egy helyes kérdését („What is the access economy?”) alá nem támasztottnak ítélte, mert a fogalom a 38 603. karakternél áll. A teljes dokumentumos konfigurációk

alátámasztottsága a hosszú dokumentumokon ezért alsó becslés. A szervermodell a naiv RAG-gal formailag hibátlan, de tartalmilag üres kérdéseket is írt („Hány szóból áll a »Link to Learning« cím?"), amelyeket a bíró 5/5-re pontozott – a szivárgáshoz hasonlóan ez is a mérést felfelé torzító hibamód.

A kis minta miatt a Holm-korrekció után egyik összevetés sem szignifikáns. A fő vizsgálat ezért 30 dokumentumon futott, és – mivel a Qwen2.5-7B a magyar szövegeken nyelvtani hibákat is vétett – két másik helyi modellel: egy magyarra továbbtanított 8B modellel (PULI-LlumiX) és egy nagyobb, 12B paraméteres többnyelvű modellel (Gemma 3).

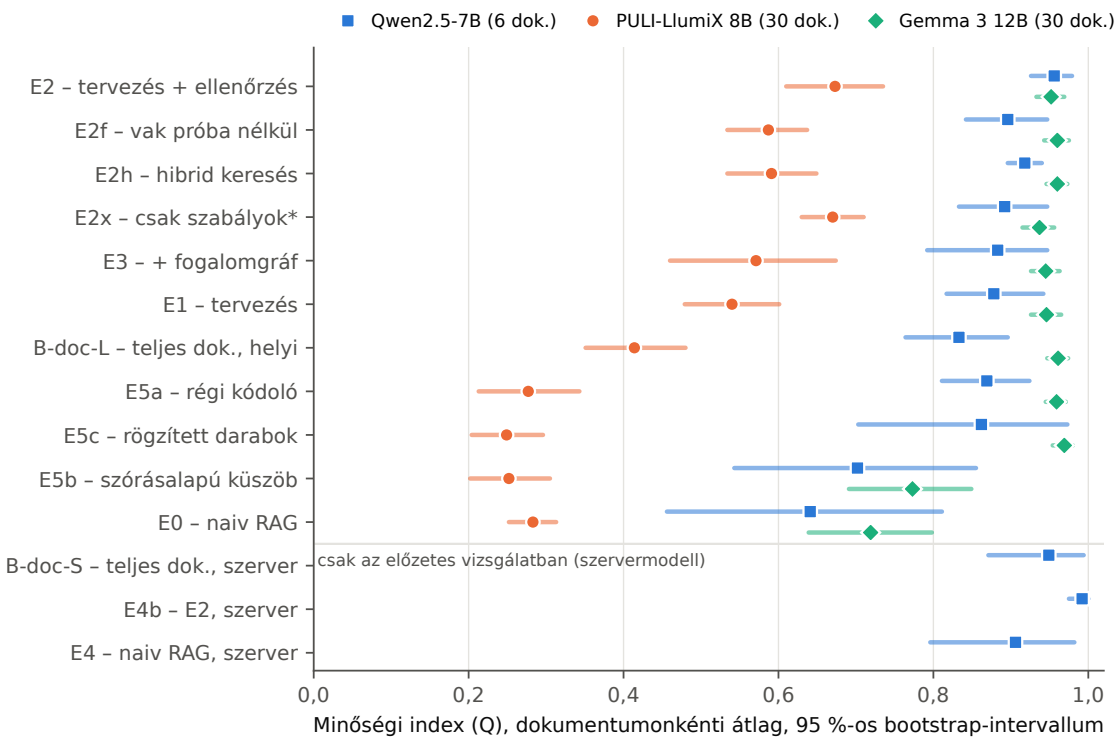
5.3. A fő vizsgálat: összesített eredmények

A fő vizsgálat 30 dokumentumon (24 magyar Wikipédia-cikk, három saját magyar és három angol OpenStax-szöveg) futtatta a 11 helyi konfigurációt mindkét modellel. A PULI-LlumiX 8B 2 935, a Gemma 3 12B 6 212 kérdést írt; a különbség oka, hogy a PULI az egyhívásos konfigurációkban a kért tíz kérdés töredékét adta vissza (5.6. szakasz). Az 5.2. táblázat a konfigurációnkénti eredményeket, az 5.1. ábra a minőségi indexet mindhárom helyi modellre mutatja.

5.2. táblázat. A fő vizsgálat eredményei (30 dokumentum). Q: minőségi index 95 %-os bootstrap-intervallummal; F, G, V: formai megfelelés, alátámasztottság, disztraktor-validitás (%); Min.: a leggyengébb dokumentum Q-ja; Perc, Hívás: medián futási idő és modellhívás vizsgánként. *Az E2x-ben a második modell ellenőrző hívásai hibával tértek vissza, így csak a szabályalapú ellenőrzés futott.

	PULI-LlumiX 8B					Gemma 3 12B						
Konfig.	Q	95 % CI	F	G	V	Q	95 % CI	F	G	V	Perc	Hívás
B-doc-L	0,41	0,35–0,48	3	48	32	0,96	0,95–0,97	100	93	95	0,6	1
E0	0,28	0,25–0,31	1	27	9	0,72	0,64–0,80	100	46	57	0,6	1
E1	0,54	0,48–0,60	47	58	49	0,95	0,93–0,96	100	88	93	2,9	14
E2	**0,67**	0,61–0,73	87	52	66	0,95	0,93–0,97	100	86	96	4,9	43
E3	0,57	0,46–0,67	80	67	60	0,94	0,93–0,96	100	92	90	6,1	56
E2f	0,59	0,53–0,64	80	47	55	0,96	0,94–0,97	100	91	96	3,9	28
E2h	0,59	0,53–0,65	77	47	59	0,96	0,95–0,97	100	92	95	4,6	42
E2x*	0,67	0,63–0,71	93	48	64	0,94	0,92–0,96	100	87	94	7,4	15
E5a	0,28	0,21–0,34	6	41	32	0,96	0,94–0,97	98	96	93	0,6	1
E5b	0,25	0,20–0,30	4	29	18	0,77	0,69–0,85	100	56	69	0,6	1
E5c	0,25	0,20–0,30	3	28	22	**0,97**	0,95–0,98	100	96	95	0,6	1

A két modell két egészen különböző képet ad. A **PULI-LlumiX** egyhívásos konfigurációi szinte teljesen elbuknak (formai megfelelés 1–6 %, $Q \leq 0{,}41$), és a módszer lépései fokozatosan javítanak: a naiv RAG-hoz képest a tervezés 0,28-ról 0,54-re, az ellenőrzés 0,67-re emeli a minőségi indexet, a teljes dokumentumos promptnál (0,41) is jóval jobb eredménnyel. A **Gemma 3 12B** viszont szinte minden konfigurációban 0,94–0,97 között teljesít: a teljes dokumentumos prompt (0,96) és a rögzített darabolású naiv RAG (E5c, 0,97) ugyanolyan jó, mint a tervezett



5.1. ábra. A minőségi index konfigurációnként és generátormodellenként, 95 %-os bootstrap-intervallummal. A Qwen2.5-7B az előzetes vizsgálat hat dokumentumán, a másik két modell a fő vizsgálat 30 dokumentumán. A szerveres konfigurációk csak az előzetes vizsgálatban futottak.

folyamat (0,95). Ennél a modellnél csak a szemantikus darabolású naiv változatok maradnak el (E0: 0,72, E5b: 0,77).

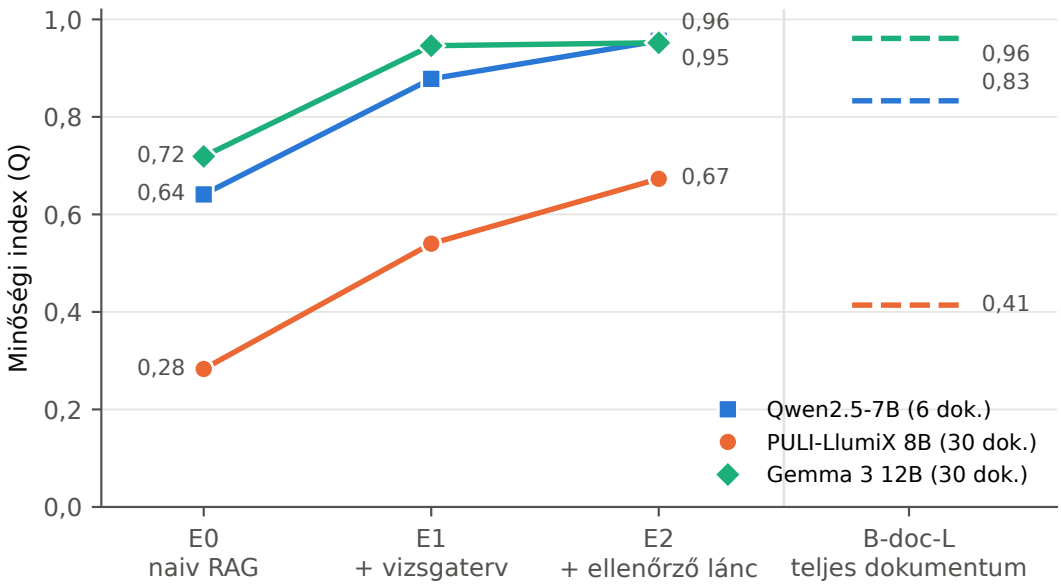
Az 5.2. ábra a három helyi modell lépésenkénti útját mutatja. A tervezés minden modellnél nagyot lép előre a naiv RAG-hoz képest; az ellenőrzés a gyengébb modelleknél (PULI, Qwen) ad hozzá, a Gemmánál nem; és a teljes dokumentumos prompt annál jobb, minél erősebb a modell.

5.4. A komponensek hatása

Az 5.3. táblázat a tervezett páros összehasonlításokat mutatja a két modellre. A 30 dokumentum már elegendő ahhoz, hogy a nagy hatások a Holm-korrekció után is szignifikánsak legyenek.

Tervezés. A vizsgaterv mindkét modellnél nagy és szignifikáns javulást hozott a naiv RAG-hoz képest (PULI: +0,26, 27 dokumentumon javított; Gemma: +0,23, 22 dokumentumon). Az előzetes vizsgálatban a Qwen2.5-7B-nél ugyanez +0,24 volt. Ez a dolgozat legerősebb, három modellen megismételt eredménye.

Ellenőrzés. Az ellenőrző lánc hatása a modelltől függ. A Qwen2.5-7B-nél öt dokumentumon javított és egyen sem rontott (+0,08), a PULI-nál +0,13 (korrigálatlan $p = 0{,}022$, Holm után 0,066), a Gemmánál gyakorlatilag nulla (+0,01). A Gemma kérdéseinek 70 %-a már az első kísérletben átment az ellenőrzésen (5.7. szakasz), így a láncnak alig van mit javítania. A



5.2. ábra. A minőségi index a módszer lépései szerint (naiv RAG → vizsgaterv → ellenőrző lánc) és a teljes dokumentumos prompt (szaggatott vonal) a három helyi modellel. Minél gyengébb a modell, annál nagyobb a módszer haszna.

5.3. táblázat. Páros Wilcoxon-próbák a dokumentumonkénti minőségi indexen (fő vizsgálat, 30 dokumentum). Δ: az új és a referencia-konfiguráció különbsége; r_{rb}: rangbiszeriális hatásméret; Jobb/rosszabb: hány dokumentumon javított, illetve rontott az új konfiguráció; $\tilde{p}$: a hat összevetésre Holm-korrigált p-érték. Félkövér: $\tilde{p} < 0{,}05$.

	PULI-LlumiX 8B				Gemma 3 12B			
Összevetés	Δ	r_{rb}	Jobb/rossz.	$\tilde{p}$	Δ	r_{rb}	Jobb/rossz.	$\tilde{p}$
Tervezés (E1–E0)	+0,26	0,97	27/2	**< 0,001**	+0,23	0,76	22/7	**0,002**
Ellenőrzés (E2–E1)	+0,13	0,48	19/10	0,066	+0,01	0,12	15/13	1,00
Teljes módszer (E2–E0)	+0,39	1,00	29/1	**< 0,001**	+0,23	0,82	23/6	**< 0,001**
Módszer vs. teljes dok. (E2–B-doc-L)	+0,26	0,83	24/5	**< 0,001**	−0,01	−0,04	16/13	1,00
Csak szabályok (E2x–E2)	0,00	0,01	14/16	0,975	−0,02	−0,12	14/15	1,00
Fogalomgráf (E3–E2)	−0,10	−0,18	15/15	0,775	−0,01	−0,20	9/16	1,00

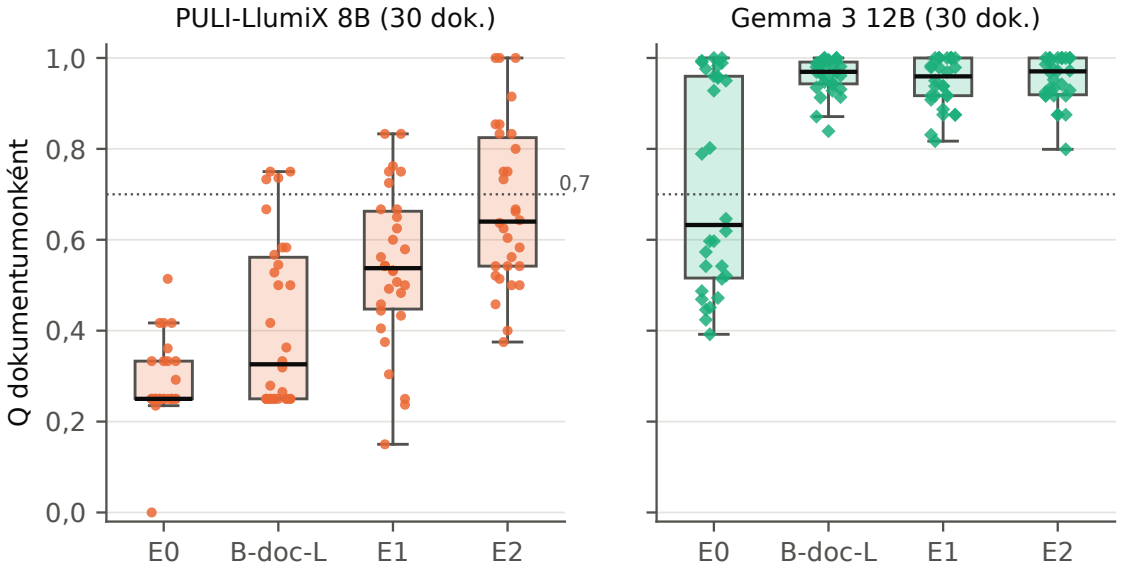
vak megoldási próba elhagyása (E2f) a PULI-nál rontott (−0,09, 21 dokumentumon rosszabb, korrigálatlan $p = 0{,}014$), a Gemmánál nem.

A független ellenőrző kérdése nyitott maradt. Az E2x-ben a második modell (Llama-3.1-8B) ellenőrző hívásai mindhárom vizsgálatban a futtató gépen hibával tértek vissza (a fő vizsgálatban 394, illetve 405 kihagyott ellenőrzés), és a folyamat – a tervezett hibakezelés szerint – ilyenkor nem blokkolja a kérdést. Az E2x így ténylegesen a „csak szabályalapú ellenőrzés” változatot mérte. Ez a PULI-nál ugyanolyan jó volt, mint a teljes E2 (0,67 vs. 0,67), a Qwen2.5-7B-nél viszont rosszabb (0,89 vs. 0,96): a PULI-nál a lánc hasznát főként a szabályok (forma, szivárgás, kérdésforma) adják, mert a modell modellalapú ítéletei a saját kérdéseiről alig informatívak.

Fogalomgráf és hibrid keresés. A fogalomgráf (E3) egyik modellnél sem érte meg a többletköltségét (PULI: −0,10, Gemma: −0,01, 56–80 hívás a 43–60 helyett), a hibrid keresés (E2h) sem javított szignifikánsan.

5.5. Robusztusság: a naiv RAG törékenysége

Egy oktató szempontjából nem az átlag, hanem az számít, mekkora a kudarc esélye egy új dokumentumon. Az 5.3. ábra a dokumentumonkénti minőségi index eloszlását mutatja. A Gemmával a tervezett konfigurációk és a teljes dokumentumos prompt egyetlen dokumentumon sem estek 0,80 alá, a naiv RAG viszont 16 dokumentumon 0,70 alatt maradt, a leggyengébben 0,39-en.

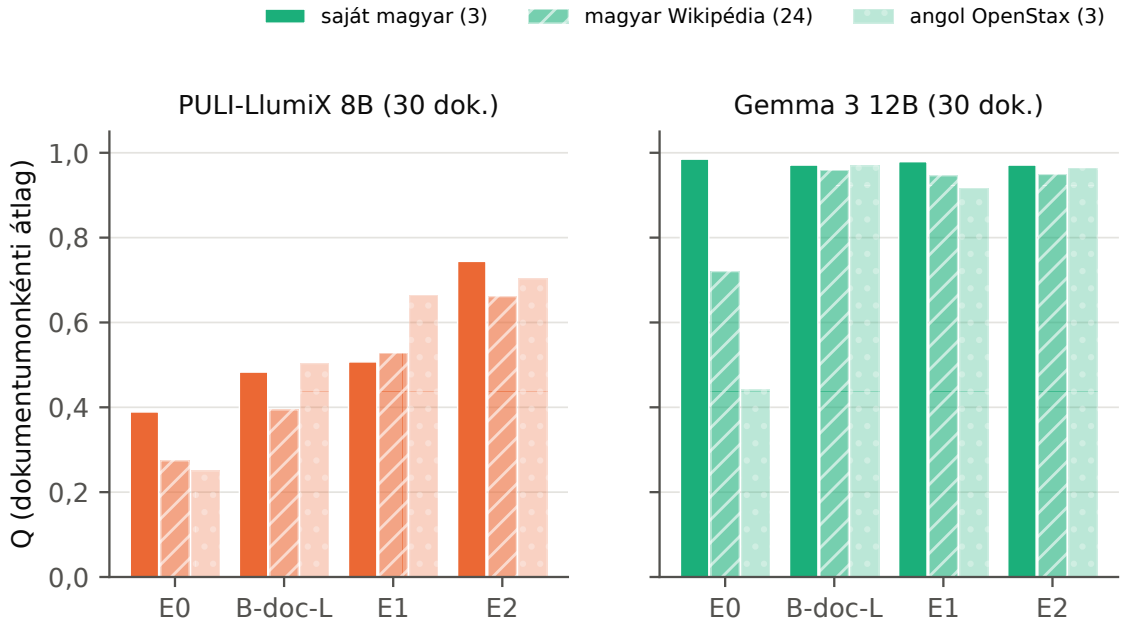


5.3. ábra. A minőségi index dokumentumonkénti eloszlása a fő vizsgálatban (pontok: egy-egy dokumentum; doboz: interkvartilis tartomány, vastag vonal: medián). A pontozott vonal a 0,70-es szint.

A naiv RAG törékenységének oka nem a keresés: a referenciakérdésekkel végzett próbában az igazoló mondatot tartalmazó darab a kérdések 86 %-ánál az első három találat között volt. A hiba a *lekérdezésben* van, amely nem tartalmi kérdés, hanem egy feladatleírás („10 feleletválasztós kérdés, könnyű nehézség”). A Wikipédia-cikkek és az OpenStax-szövegek rövid szerkezeti sorokat (címeket, „Link to Learning”-dobozokat) tartalmaznak, amelyeket a szemantikus darabolás minimális darabméret nélkül önálló darabbá vág, és a tartalom nélküli lekérdezés éppen ezekhez a tartalom nélküli darabokhoz áll a legközelebb. A Gemma ilyenkor magáról a feladatról kérdez: egy, a sclerosis multiplexről szóló cikkből például ezt a kérdést írta: „Mi a nehézségi szint a vizsgának?”, kulcsa: „Kezdő (könnyű).” A rögzített, 800 karakteres darabolás (E5c) ilyen darabot nem hoz létre, ezért a naiv RAG azonos modellel 0,72 helyett 0,97-et ért el; a tervezett folyamat pedig kérdéshelyenként a tervezett fogalommal keres, így a darabolás e részlete nem befolyásolja.

Az 5.4. ábra ugyanezt forrásonként mutatja: a Gemma naiv RAG-ja a három rövid saját

szövegen kiváló (0,99), a Wikipédia-cikkeken 0,72-re, az angol tankönyvi szövegeken 0,44-re esik, miközben a többi konfiguráció minden forráson 0,92 felett marad. A magyar és angol dokumentumok között a tervezett és a teljes dokumentumos konfigurációk nem mutatnak lényeges különbséget; mivel azonban az angol részhalmaz mindössze három dokumentum, ez nyelvi összevetésre nem elegendő.



5.4. ábra. A minőségi index dokumentumforrásonként a fő vizsgálat négy fő konfigurációjában. Zárójelben a dokumentumok száma.

5.6. Miért bukik el a PULI-LlumiX?

A magyarra továbbtanított modelltől azt vártuk, hogy a magyar dokumentumokon jobb lesz a többnyelvű modelleknél. Ehelyett a fő vizsgálat leggyengébb modellje lett, és ennek oka nem a nyelv, hanem az *utasításkövetés*. Egyetlen hívásban a PULI a kért tíz kérdés helyett átlagosan 1,8-at (E0), illetve 5,6-ot (B-doc-L) adott vissza, így a vizsgák 97–99 %-a formailag hibás volt; a darabolási változatok vizsgáinak 15–29 %-ában a válasz nem is volt értelmezhető JSON. A tervezett folyamat ezt megkerüli, mert egy hívásban csak egy kérdést kér, de a kérdések minősége így is gyenge marad: az ellenőrzés nélküli E1 kérdéseinek 34 %-a szivárogtatja a kulcsot, 45 %-ának törzse nem kérdés, a kérdéspárok 62 %-a pedig közel azonos. A modell jellemzően a forrás egy mondatát másolja be kérdésként és kulcsként is, ugyanazt a hibát mutatva, amelyet a pilot a Qwen-ellenőrző visszacsatolásánál talált. A *Nürnbergi per* cikkből például a „kérdés” a forrás egy kijelentő mondata volt, és a négy válaszlehetőség közül kettő ugyanez a mondat; a bíró helyességi pontszáma 1/5 lett.

Az ellenőrző lánc a szivárgást (34 %→12 %) és a nem kérdő törzseket (45 %→22 %) mérsékli, de nem szünteti meg, és a megvalósult Bloom-szint gyakorlatilag mindig az emlékezés maradt.

Az E1 futása emellett instabil volt: 103 modellhívás időtúllépéssel vagy hibával zárult, és egy vizsga medián ideje 96 perc lett (a Gemmánál 2,9 perc). A tanulság gyakorlati: egy magyar nyelvi továbbtanítás önmagában nem teszi a modellt alkalmassá strukturált, formátumkövető feladatra; a modellválasztásnál az utasításkövetés legalább olyan fontos, mint a nyelvi teljesítmény.

5.7. Az ellenőrző működése

Az 5.4. táblázat az E2 ellenőrző láncának működését hasonlítja össze a három modellnél. Az ellenőrzőt bináris osztályozóként is értékeljük: az általa átengedett kérdéseket a bíró alátámasztottsági ítéletével vetjük össze (4.8. szakasz).

5.4. táblázat. Az E2 ellenőrző láncának működése modellenként. Első kísérlet: az első próbálkozásra elfogadott kérdéshelyek aránya; Átengedett: az ellenőrzöttként elfogadott kérdések aránya; pontosság, felidézés és Cohen-κ a bíró alátámasztottsági ítéletéhez képest (csak a bírált kérdéseken). Az elutasítási okok az összes kísérletre vonatkoznak.

Modell	Első kísérlet	Kísérlet/ kérdéshely	Átengedett	Pontosság	Felidézés	κ	Leggyakoribb elutasítási okok
Qwen2.5-7B	23 %	1,8	52 %	0,94	0,54	0,08	duplikátum, igaz lehet egy disztraktor, nem kérdés
PULI-LlumiX 8B	5 %	2,0	19 %	0,56	0,21	0,05	duplikátum, forma, nem kérdés, vak eltérés, szivárgás
Gemma 3 12B	70 %	1,3	95 %	0,88	0,95	0,00	duplikátum, vak eltérés

A táblázatból három tanulság olvasható ki. (1) A leggyakoribb elutasítási ok minden modellnél a *duplikátum*: tíz egymástól eltérő kérdés megírása egy rövid szövegből a kis modellek számára nehéz. (2) Az ellenőrző terhelése a modell gyengeségét tükrözi: a Gemmánál a kérdéshelyek 70 %-a elsőre átment, a PULI-nál 5 %-a. (3) Az ellenőrző kérdésenkénti ítélete egyik modellnél sem egyezik a bíróéval a véletlennél jobban ($\kappa = 0{,}00$–$0{,}08$). A Qwen ellenőrzője pontos, de szigorú: amit átenged, az 94 %-ban alátámasztott, de a jó kérdések közel felét is megjelöli; a Gemma szinte mindent átenged. Az ellenőrzés tehát a vizsga egészét javíthatja (szűrőként és „második esélyként”), de jelzése kérdésenként nem megbízható ítélet – ennek gyakorlati következményét a 6. fejezet tárgyalja.

5.8. Lefedettség, kognitív szint és felhasználhatóság

Lefedettség. A vizsgaterv lefedettségi előnye a naiv RAG-gal szemben mindkét modellnél megjelent (Gemma: 24 %→43 %, PULI: 10 %→21 %), a teljes dokumentumos prompttal szemben viszont nem: a Gemma teljes dokumentumos vizsgái ugyanannyi darabot érintettek (43 %), mint a tervezettek. A kérdések egy szakaszra tömörülése tehát a naiv RAG sajátja, nem a tel-

jes dokumentumos prompté. A lefedettség felső korlátja tíz kérdésnél $10/K$, ezért a hosszú dokumentumokon a mutató egyik módszert sem különbözteti meg.

Kognitív szint. A vizsgaterv a fő vizsgálatban a kérdések 43 %-át tervezte magasabb rendű (alkalmazás, elemzés, értékelés) szintre. A bíró szerint ezekből a Gemmánál 7–11 %, a PULI-nál gyakorlatilag 0 % valósult meg, és a tervezett és a megvalósult szint a Gemmánál a kérdések 16–20 %-ánál egyezett. Ugyanez az előzetes vizsgálatban a Qwennél 35 %, a szervermodellnél 62 % volt. A Bloom-kiosztás tehát a *kért* szintet garantálja; a *megvalósult* szint egy kis modellnél ellenőrizendő.

Felhasználhatóság. A bíró „felhasználnám” ítélete (bekerülne-e a kérdés egy valódi dolgozatba legfeljebb apró javítással) a Gemmánál a tervezett konfigurációkban 85–88 %, a teljes dokumentumos promptnál 83 %, a naiv RAG-nál 42 %; a PULI-nál legfeljebb 29 % (E2). Az előzetes vizsgálatban a Qwen E2 kérdéseinek 80 %-a, a szervermodell E4b kérdéseinek 98 %-a kapott ilyen ítéletet. A tervezett konfigurációk kérdései hosszabbak (a törzs átlagosan kb. 120 karakter, a Gemma teljes dokumentumos kérdéseinél 63): a tervezett folyamat jellemzően magyarázó, összefüggést kérdező törzset ír.

5.9. Erőforrásigény

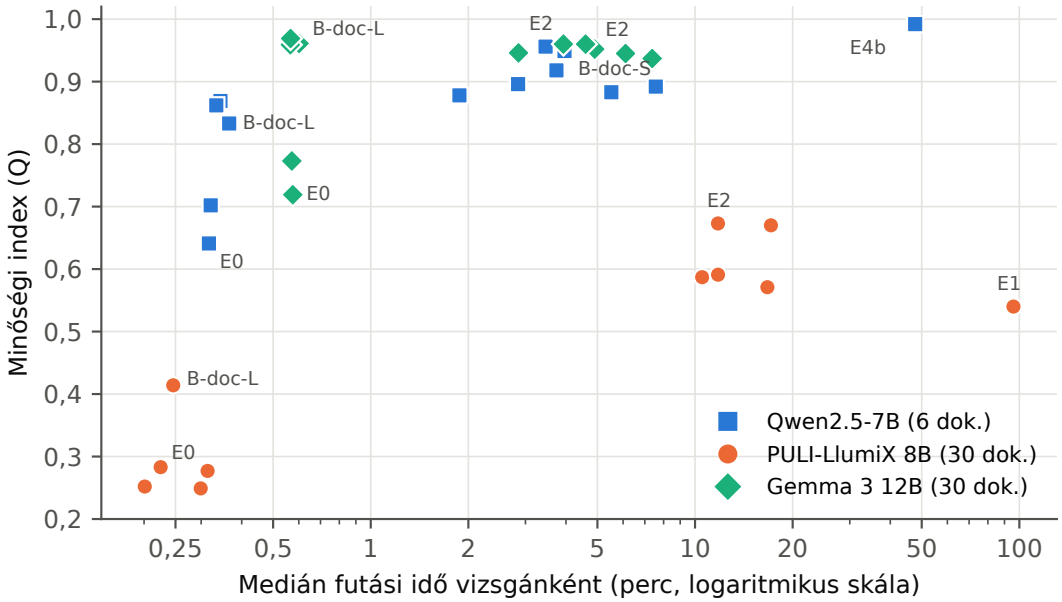
Az 5.5. táblázat a három helyi modell mért memóriaigényét és a fő konfigurációk futási idejét foglalja össze; az 5.5. ábra a minőséget a futási idő függvényében mutatja.

5.5. táblázat. A helyi modellek erőforrásigénye. Modell: az Ollama által jelentett foglalás 8 192 tokenes kontextusnál (MiB); Kártya: a szükséges fogyasztói GPU az asztali környezet 1,6–1,9 GiB-os foglalása mellett (a pilot mérése); Perc: medián futási idő vizsgánként RTX A4500-on.

Modell	Modell (MiB)	Kártya	B-doc-L perc	E1 perc	E2 perc
Qwen2.5-7B (Q4_K_M)	4 896	8 GB	0,4	1,9	3,5
PULI-LlumiX 8B (Q4_K_M)	5 676	8 GB	0,2	95,9*	11,8
Gemma 3 12B (Q4_K_M)	7 977	12 GB	0,6	2,9	4,9

* Időtúllépések miatt (5.6. szakasz).

A Qwen2.5-7B és a PULI-LlumiX 8B egy 8 GB-os kártyán is elfér (a pilot asztali gépén, RTX 3070 Ti mellett a legrosszabb esetben is kb. 380 MiB tartalék maradt); a Gemma 3 12B 7,8 GiB-os foglalása az asztali környezettel együtt kb. 9,5–10 GiB, vagyis egy 12 GB-os fogyasztói kártyát igényel. A 16 384 tokenes teljes dokumentumos konfigurációnál a Qwen foglalása 464 MiB-tal nőtt, ami nagyságrendben egyezik a (3.1) becslés 448 MiB-os KV-többletével. Futási időben a tervezett folyamat egy nagyságrenddel lassabb az egyhívásos megoldásoknál (3–5 perc 20–40 másodperc helyett), ami egy tízkérdéses vizsgánál elfogadható várakozás.



5.5. ábra. Minőség és futási idő. Az egyhívásos konfigurációk gyorsak, de minőségük a modelltől és a darabolástól függően erősen ingadozik; a tervezett konfigurációk lassabbak, de minden modellnél a legjobbak között vannak.

5.10. A mérés megbízhatósága

Hiányzó bírói ítéletek. A fő vizsgálat pontozása során a bíró szervere a kérések egy részére hibával válaszolt (HTTP 400, újrapróbálás után is), ezért a kérdések 28–65 %-ánál (az egyhívásos PULI-konfigurációknál 70–100 %-ánál) van meg az áttekintő ítélet; az előzetes vizsgálatban ilyen hiba nem fordult elő. A hiba a dokumentum hosszától és forrásától független volt, így a mutatók a sikeresen bírált kérdéseken számolt, kisebb mintájú becslések (konfigurációnként 130–580 kérdés). Azt, hogy ez a következtetéseket nem torzítja, érzékenységvizsgálattal ellenőriztük: ha az alátámasztottságot, a vak megoldhatóságot és a disztraktor-validitást a vizsgák helyett dokumentumonként, az összes bírált kérdésre összevonva számoljuk, a minőségi index a Gemma minden konfigurációjában és a PULI tervezett konfigurációiban legfeljebb 0,01-dal tér el; a sorrend és a szignifikáns különbségek nem változnak.

Oktatói értékelés. A bíró oktatói validálása (4.7. szakasz) a dolgozat lezárásakor folyamatban van. [Ide kerül: az oktatók egymás közötti egyezése (Krippendorff-α), a bíró és az oktatók egyezése szempontonként (Spearman, α), a „felhasználnám” ítéletek Cohen-κ-ja, valamint az oktatók szerinti minőség konfigurációnként.]

6. Az eredmények értékelése

Ebben a fejezetben az eredményeket a hipotézisek szempontjából értékeljük, majd megvizsgáljuk, mikor éri meg a módszer, mit tanultunk az ellenőrzésről és a mérésről, és milyen gyakorlati ajánlás fogalmazható meg. A fejezetet az érvényességet veszélyeztető tényezők zárják.

6.1. A hipotézisek értékelése

6.1. táblázat. A hipotézisek és a vizsgálatok bizonyítéka. Fő: a 30 dokumentumos fő vizsgálat (PULI-LlumiX 8B és Gemma 3 12B); Előzetes: a hat dokumentumos vizsgálat (Qwen2.5-7B és a szervermodell).

Hipotézis	Bizonyíték	Értékelés
1. Tervezés	Fő: Q +0,26 (PULI) és +0,23 (Gemma), Holm után $\tilde{p} < 0{,}001$, illetve 0,002; alátámasztottság 27→58, illetve 46→88 %; lefedettség 10→23, illetve 24→42 %; a forma nem romlott. Előzetes: +0,24 (nem szignifikáns, 6 dok.)	**támogatott**
2. Ellenőrzés	Q: Qwen +0,08 (5 dok. javult, 0 romlott), PULI +0,13 ($\tilde{p} = 0{,}066$), Gemma +0,01; az alátámasztottság csak a Qwennél nőtt; $\kappa = 0{,}00$–$0{,}08$ minden modellnél	**részben**: modellfüggő; az egyezésre vonatkozó rész nem teljesült
3. Lefedettség	Gemma: teljes dok. 43 % vs. E2 43 %; Qwen: 45 vs. 44 %; a tömörülés a naiv RAG sajátja (10–25 %)	**nem támogatott**
4. Helyi vs. szerver	Előzetes: a közös dokumentumokon E2 0,95 vs. E4 0,93, és E2 0,96 vs. B-doc-S 0,95; a Gemma E2 ugyanezen a hat dokumentumon 0,97; a modellek 4,8–7,8 GiB-ot foglalnak. A PULI-val nem teljesül (0,72)	**támogatott** (leíró, modellfüggő)
5. Mérés	Az oktatói értékelés folyamatban (5.10. szakasz)	**nyitott**

1. hipotézis. A vizsgaterv a dolgozat legerősebb és legjobban megismételt eredménye: három különböző helyi modellel, két vizsgálatban azonos irányú, és a fő vizsgálatban mindkét modellnél szignifikáns. A javulás forrása elsősorban az, hogy a kérdéshelyenkénti keresés a tervezett fogalommal, vagyis tartalmi lekérdezéssel dolgozik a naiv RAG feladatleírása helyett.

2. hipotézis. Az ellenőrzés hatása a modell erősségétől függ. A Qwennél minden tartalmi mutatón javított, a PULI-nál a minőségi index főként a forma és a disztraktorok javulásából nőtt (az alátámasztottság nem), a Gemmánál pedig nem volt mit javítania, mert a kérdések 70 %-a elsőre átment. A hipotézis második fele – hogy az ellenőrző ítélete egyezik a bíróéval – egyik modellnél sem teljesült érdemben. Ezt az eredményt fontosnak tartjuk: egy önellenőrző kis modell kérdésenkénti jelzése nem helyettesíti sem a független bírót, sem az oktatót.

3. hipotézis. A teljes dokumentumot kapó modell nem tömöríti a kérdéseit a dokumentum egy részére; a P2 lefedettségi problémája a vizsgált hosszúságú dokumentumokon a naiv visszake-

resés sajátja. Ehhez hozzá kell tenni, hogy tíz kérdésnél a lefedettség felső korlátja a hosszú dokumentumokon minden konfigurációt egyformán korlátoz.

4. és 5. hipotézis. A helyi, tervezett és ellenőrzött folyamat a vizsgált dokumentumokon elérte a szervermodell naiv és teljes dokumentumos változatát, fogyasztói GPU-n. Az eredmény azonban nem minden helyi modellre igaz: a PULI-LlumiX-szel nem. A bíró emberi validálása folyamatban van; a vizsgálatok addig is több bírói vakfoltot tártak fel (6.3. szakasz), ezért a bíró ítéleteit relatív, a konfigurációk összevetésére szolgáló mérésként kezeljük.

6.2. Mikor éri meg a módszer?

A dolgozat kiinduló állítása az volt, hogy a vizsgagenerálás megbízhatóságát nem egy jobb prompt, hanem a feladat ellenőrizhető lépésekre bontása biztosítja. A három modellen végzett mérés ezt az állítást pontosítja: *a módszer haszna fordítottan arányos a modell képességével* (5.2. ábra).

- **Gyenge, rossz formátumkövető modellnél** (PULI-LlumiX 8B) a teljes dokumentumos prompt és a naiv RAG használhatatlan ($Q \leq 0{,}41$), és csak a felbontás ad értékelhető vizsgát (0,67). A módszer itt a formátumot és a darabszámot is kikényszeríti, amit a modell magától nem tart be.
- **Közepes modellnél** (Qwen2.5-7B) a módszer emeli a minőséget a szervermodell szintjére ($0{,}83 \rightarrow 0{,}96$), egy 8 GB-os kártyán.
- **Erős helyi modellnél** (Gemma 3 12B) a teljes dokumentumos prompt is 0,96-ot ér el, a módszer pedig átlagban nem ad hozzá. Itt a felbontás értéke nem az átlagos minőség, hanem az, amit egy prompt nem tud: minden kérdés a saját forrásdarabjára hivatkozik, így az oktató a szerkesztőben egy kattintással ellenőrizheti (P1); a darabszám, a típus és a Bloom-szint a terv része, nem kérés (P2); és az eredmény nem függ a darabolás részleteitől.

Két megfigyelés minden modellnél azonos. Egyrészt *a naiv RAG törékeny*: a feladatleírásból képzett lekérdezés a tartalom nélküli darabokhoz áll a legközelebb, és ugyanaz a modell a darabolás egy apró részletén múlóan 0,72 vagy 0,97 minőséget ad. Ez a hiba a gyakorlatban észrevétlen maradna, mert a kérdések formailag hibátlanok. Másrészt *a modellválasztás nagyobb hatású, mint bármely komponens*: ugyanaz a teljes módszer a PULI-val 0,67-et, a Gemmával 0,95-öt ad. A PULI-LlumiX eredménye külön tanulság: a magyar nyelvi továbbtanítás nem pótolja az utasításkövetést, és egy strukturált feladatra a többnyelvű, jól formátumkövető modell a jobb választás magyar dokumentumokon is.

6.3. Mit tanultunk az ellenőrzésről és a mérésről?

Az ellenőrzés a vizsgát javítja, nem a jelzést. Az ellenőrző lánc jelzései egyenként zajosak, de az újrapróbálkozás miatt összegződnek: ha egy kérdést elutasítunk, és a generátor újat ír, az új kérdés átlagosan jobb. A lánc így szűrőként és „második esélyként” működik, nem megbízható osztályozóként – összhangban azzal, hogy a modellek önellenőrzése külső visszajelzés nélkül nem megbízható [5], de a forrásszöveggel és determinisztikus szabályokkal kombinálva javíthat. A szabályok szerepe védelmi: a szivárgás és a nem kérdő törzsek elleni szabályok nélkül a visszacsatolás a forrás szó szerinti másolása felé tolja a modellt.

A mérés a valósnál jobb eredményt mutatott volna. A munka egyik fő tanulsága módszertani: a mérés több ponton is torzított volna, ha nem ellenőrizzük. A tartalékvizsga, a csonkolt kódoló és a válaszszivárgás mind *felfelé* torzított; a szervermodell tartalmilag üres kérdéseit a bíró hibátlannak pontozta; a bíró 24 000 karakteres ablaka a hosszú dokumentumok végéről írt helyes kérdéseket *lefelé* torzította. A fő vizsgálatban két újabb csendes hiba jelent meg: a független ellenőrző (E2x) hívásai harmadszor is hibáztak, így a konfiguráció nem azt mérte, amit a neve ígért, és a bíró kéréseinek egy része hibával tért vissza. A közös vonás, hogy a mérőrendszer mindvégig „működött”: hihető számokat adott, és nem jelzett hibát. A nyelvi modellekre épülő rendszerek kiértékelésében ezért (i) a nyers kimenetek kézi átolvasása nem helyettesíthető automatizmussal, és (ii) minden csendes tartalékágat (kihagyott ellenőrzés, sikertelen bírói hívás, tartalékkérdés) a jelentésben is meg kell mutatni.

Gyakorlati következmény a felületre. Az ellenőrző jelzését az oktatónak *figyelmeztetésként* kell megmutatni („ezt a kérdést nézd meg alaposabban”), nem ítéletként: a Qwennél a megjelölt kérdések többsége valójában rendben volt, a Gemmánál pedig szinte minden kérdés átment. Ha a rendszer az átengedett kérdéseket „ellenőrzöttként” hirdetné, az automatizálási torzítás [74] miatt az oktató éppen ezeket nézné át kevésbé figyelmesen.

6.4. Gyakorlati ajánlás

- **8 GB-os kártyán** (például egy oktatói asztali gépen): a Qwen2.5-7B a teljes módszerrel (E2), vizsgánként 3–5 perc alatt, a vizsgált dokumentumokon a nagy szervermodellel azonos minőséget ad.
- **12 GB-os kártyán**: a Gemma 3 12B a tervezéssel (E1, kb. 3 perc) vagy ellenőrzéssel (E2, kb. 5 perc); rövid dokumentumnál a teljes dokumentumos prompt is jó, de ott a kérdések forrása nem követhető vissza.
- **A naiv RAG nem ajánlott**, mert minősége a darabolás részleteitől függően kiszámíthatatlan; ha mégis ezt használjuk, a darabolónak minimális darabméretet kell kapnia.
- **Modellválasztásnál** az utasítás- és formátumkövetés fontosabb a magyar nyelvi specializációnál.

– **Minden esetben** oktatói átnézés szükséges; a MIMIR ezt kötelezővé is teszi.

6.5. Az érvényességet veszélyeztető tényezők

Belső érvényesség. Az egytényezős terv a különbségeket egy-egy tényezőhöz köti, de a tényezők nem teljesen függetlenek: az ellenőrzés például a kérdések hosszát is megváltoztatja, ami közvetve hathat a bíróra. A tervezett konfigurációk egy maggal futottak, így dokumentumonkénti értékükben a futásról futásra jelentkező zaj is benne van. A három vizsgálat a rendszer kissé eltérő változatait méri, ezért összevetést csak vizsgálaton belül végeztünk. Az E2x a tervezettől eltérő konfigurációt mért, és a PULI-nál a második ellenőrző (Llama-3.1-8B) amúgy sem tartozott volna más modellcsaládba.

Konstrukciós érvényesség. A minőségi index egyenlő súlyú komponensei közül a formai megfelelés vizsgaszintű, ezért egy formai hiba nagy hatású; ezért minden komponenst külön is közlünk. A tartalmi mutatók egyetlen, oktatókkal még nem validált bíróra épülnek, amely a kérdés saját forrását látja, a dokumentum más részeiben lévő ellentmondást tehát nem. A fő vizsgálatban a bírói ítéletek egy része hiányzik; az érzékenységvizsgálat szerint ez a következtetéseket nem befolyásolja (5.10. szakasz), de a becslések pontosságát csökkenti.

Külső érvényesség. A fő vizsgálat 30 dokumentumából 27 magyar, 24 Wikipédia-cikk; az angol részhalmaz három dokumentum, így nyelvi összevetésre nem alkalmas. A szervermodell csak a hat dokumentumos előzetes vizsgálatban szerepelt. Csak feleletválasztós kérdéseket értékeltünk. A nyílt dokumentumok a modellek tanítóadatában is szerepelhettek [73], ezért a vak megoldhatóságot mindig az alátámasztottsággal együtt értelmezzük.

Következtetési érvényesség. A dokumentumszintű nemparaméteres próbák konzervatívak; a 30 dokumentum a nagy hatások kimutatására elegendő, a kicsikére (például az ellenőrzés $+0{,}01$-es hatására a Gemmánál) nem. A hat dokumentumos előzetes vizsgálat következtetései leíró jellegűek.

Etikai és jogi szempontok. A rendszer hallgatói adatot nem kezel, a feltöltött dokumentumok a feladat végén törlődnek, minden kérdés oktatói jóváhagyást igényel, az export pedig feltünteti az MI-közreműködést [76]. A kiértékelő dokumentumok saját vagy nyílt licencű művek, a szerveres modellek és a bíró az egyetem saját infrastruktúráján futottak, így a kiértékelés során sem került tananyag külső szolgáltatóhoz.

7. Összefoglalás és további munka

7.1. Összefoglalás

Dolgozatunkban a vizsgakérdés-generálás négy gyakorlati problémájából indultunk ki: a kulcsok helyessége nincs ellenőrizve (P1), a vizsga szerkezete és lefedettsége nincs kontrollálva (P2), a tananyag egy harmadik félhez kerül (P3), és a módszer nagy, távoli modellekre támaszkodik (P4). Ezekre a MIMIR rendszerrel válaszoltunk, amely az egyetlen promptot ellenőrizhető lépésekre bontott módszerrel váltja fel, és egyetlen fogyasztói GPU-n, adatmegőrzés nélkül fut. A módszert egy nyílt kiértékelő keretrendszerrel, 14 konfigurációval, három vizsgálatban és három helyi modellel mértük.

A kutatási kérdésekre a következő válaszokat adhatjuk.

RQ1. A strukturált generálás minden vizsgált modellnél jelentősen jobb, mint a naiv visszakeresés (a fő vizsgálatban $+0{,}23$ és $+0{,}39$ közötti javulás, Holm-korrekció után is szignifikánsan), és jobb lefedettséget ad. A teljes dokumentumos prompttal szemben az előny a modelltől függ: a gyengébb modelleknél nagy ($0{,}41 \rightarrow 0{,}67$ és $0{,}83 \rightarrow 0{,}96$), az erős Gemma 3 12B-nél nincs (0,96 és 0,95), és a lefedettség sem jobb.

RQ2. A legnagyobb hatású komponens a vizsgaterv, amely kérdéshelyenként tartalmi lekérdezéssel keres. Az ellenőrző lánc a gyengébb modelleknél javít ($+0{,}08$, $+0{,}13$), az erősnél nem, és kérdésenkénti ítélete a bíróéval a véletlen szintjén egyezik. A fogalomgráf és a hibrid keresés a többletköltségéért nem javított. A naiv RAG minőségét a darabolás apró részletei is drámaian befolyásolták.

RQ3. A helyi, tervezett folyamat a vizsgált dokumentumokon elérte a szervermodell naiv és teljes dokumentumos változatát, 8 GB-os (Qwen2.5-7B) vagy 12 GB-os (Gemma 3 12B) fogyasztói kártyán, vizsgánként 3–5 perc alatt. Az eredmény erősen függ a helyi modelltől: a magyarra továbbtanított PULI-LlumiX 8B a formátumkövetésben elbukott, így a magyar dokumentumokon is a többnyelvű modellek voltak jobbak.

Módszertani eredményként a vizsgálatok több olyan hibát tártak fel, amely a mérést a valósnál jobbnak mutatta volna – köztük a válaszszivárgást, amelyben az ellenőrző visszacsatolása a helyes választ a kérdés szövegébe írta. A tanulság általános: egy nyelvi modellekre épülő rendszer kiértékelésében minden csendes tartalékágat láthatóvá kell tenni, és a nyers kimeneteket kézzel is át kell olvasni.

7.2. További munka

– **A bíró validálása.** Az oktatói értékelés befejezése és elemzése (4.7. szakasz), amely eldönti az 5. hipotézist.

– **A hiányzó mérések pótlása.** A sikertelen bírói hívások újrapontozása, a független ellenőrző (E2x) újramérése úgy, hogy a keretrendszer a kihagyott ellenőrzést hibaként jelezze, és a szervermodell futtatása a fő vizsgálat 30 dokumentumán.
– **Adaptív folyamat.** Mivel a módszer haszna a modelltől függ, a rendszer a modell első kísérleteinek sikerarányából eldöntheti, szükség van-e ellenőrzésre; erős modellnél így a futási idő a töredékére csökkenthető.
– **Megbízhatóbb ellenőrző és Bloom-szint.** Természetes nyelvi következtetésre tanított osztályozó [49] a kérdésenkénti jelzéshez, és egy szintosztályozó lépés, mert a kis modellek a kért magasabb rendű szintet ritkán valósították meg.
– **Tanórai kipróbálás.** Az igaz–hamis és a kifejtős kérdések kiértékelése, valamint a generált vizsgák tételelemzése (nehézség, megkülönböztető erő, nem működő disztraktorok [10]) valós hallgatói válaszokon.

A MIMIR nyílt forráskódú: a kód, a konfigurációk, az adatkészlet-leírások és a vizsgálatok nyers kimenetei a reprodukáláshoz rendelkezésre állnak.

Irodalomjegyzék

[1] T. M. Haladyna, S. M. Downing és M. C. Rodriguez. „A Review of Multiple-Choice Item-Writing Guidelines for Classroom Assessment”. *Applied Measurement in Education* 15.3 (2002), 309–333. old.

[2] M. J. Gierl, O. Bulut, Q. Guo és X. Zhang. „Developing, Analyzing, and Using Distractors for Multiple-Choice Tests in Education: A Comprehensive Review”. *Review of Educational Research* 87.6 (2017), 1082–1116. old.

[3] G. Kurdi, J. Leo, B. Parsia, U. Sattler és S. Al-Emari. „A Systematic Review of Automatic Question Generation for Educational Purposes”. *International Journal of Artificial Intelligence in Education* 30.1 (2020), 121–204. old. DOI: 10.1007/s40593-019-00186-y.

[4] Z. Ji, N. Lee, R. Frieske és tsai. „Survey of Hallucination in Natural Language Generation”. *ACM Computing Surveys* 55.12 (2023), 1–38. old.

[5] J. Huang, X. Chen, S. Mishra és tsai. „Large Language Models Cannot Self-Correct Reasoning Yet”. *International Conference on Learning Representations (ICLR)*. 2024.

[6] N. F. Liu, K. Lin, J. Hewitt és tsai. „Lost in the Middle: How Language Models Use Long Contexts”. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. old.

[7] Európai Parlament és a Tanács. *Az Európai Parlament és a Tanács (EU) 2016/679 rendelete (általános adatvédelmi rendelet)*. Az Európai Unió Hivatalos Lapja, L 119. 2016.

[8] S. M. Downing. „The Effects of Violating Standard Item Writing Principles on Tests and Students: The Consequences of Using Flawed Test Items on Achievement Examinations in Medical Education”. *Advances in Health Sciences Education* 10.2 (2005), 133–143. old. DOI: 10.1007/s10459-004-4019-5.

[9] M. C. Rodriguez. „Three Options Are Optimal for Multiple-Choice Items: A Meta-Analysis of 80 Years of Research”. *Educational Measurement: Issues and Practice* 24.2 (2005), 3–13. old. DOI: 10.1111/j.1745-3992.2005.00006.x.

[10] M. Tarrant, J. Ware és A. M. Mohammed. „An Assessment of Functioning and Non-Functioning Distractors in Multiple-Choice Questions: A Descriptive Analysis”. *BMC Medical Education* 9 (2009), 40. old. DOI: 10.1186/1472-6920-9-40.

[11] L. W. Anderson és D. R. Krathwohl, szerk. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York: Longman, 2001.

[12] D. Scully. „Constructing Multiple-Choice Items to Measure Higher-Order Thinking”. *Practical Assessment, Research, and Evaluation* 22.4 (2017).

[13] R. Mitkov és L. A. Ha. „Computer-Aided Generation of Multiple-Choice Tests”. *Proceedings of the HLT-NAACL 03 Workshop on Building Educational Applications Using Natural Language Processing*. 2003, 17–22. old.

[14] M. Heilman és N. A. Smith. „Good Question! Statistical Ranking for Question Generation”. *Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics*. 2010, 609–617. old.

[15] P. Rajpurkar, J. Zhang, K. Lopyrev és P. Liang. „SQuAD: 100,000+ Questions for Machine Comprehension of Text”. *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. 2016, 2383–2392. old.

[16] X. Du, J. Shao és C. Cardie. „Learning to Ask: Neural Question Generation for Reading Comprehension”. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*. 2017, 1342–1352. old.

[17] E. Alhazmi, Q. Z. Sheng, W. E. Zhang, M. Zaib és A. Alhazmi. „Distractor Generation in Multiple-Choice Tasks: A Survey of Methods, Datasets, and Evaluation”. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, 14437–14458. old.

[18] Z. Wang, J. Valdez, D. Basu Mallick és R. G. Baraniuk. „Towards Human-Like Educational Question Generation with Large Language Models”. *Artificial Intelligence in Education (AIED 2022)*. 13355. köt. Lecture Notes in Computer Science. Springer, 2022, 153–166. old.

[19] J. Doughty, Z. Wan, A. Bompelli, J. Qayum, T. Wang, J. Zhang és tsai. „A Comparative Study of AI-Generated (GPT-4) and Human-crafted MCQs in Programming Education”. *Proceedings of the 26th Australasian Computing Education Conference*. 2024, 114–123. old.

[20] S. K. Bitew, J. Deleu, C. Develder és T. Demeester. *Distractor Generation for Multiple-Choice Questions with Predictive Prompting and Large Language Models*. arXiv:2307.16338. 2023.

[21] A. Hadifar, S. K. Bitew, J. Deleu, C. Develder és T. Demeester. „EduQG: A Multi-Format Multiple-Choice Dataset for the Educational Domain”. *IEEE Access* 11 (2023), 20885–20896. old.

[22] F. Fawzi, S. Amini és S. Bulathwela. „Small Generative Language Models for Educational Question Generation”. *NeurIPS 2023 Workshop on Generative AI for Education (GAIED)*. 2023.

[23] N. Mulla és P. Gharpure. „Automatic Question Generation: A Review of Methodologies, Datasets, Evaluation Metrics, and Applications”. *Progress in Artificial Intelligence* 12.1 (2023), 1–32. old.

[24] A. Vaswani, N. Shazeer, N. Parmar és tsai. „Attention Is All You Need”. *Advances in Neural Information Processing Systems*. 30. köt. 2017.

[25] J. Devlin, M.-W. Chang, K. Lee és K. Toutanova. „BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*. 2019, 4171–4186. old.

[26] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal és tsai. „Language Models are Few-Shot Learners”. *Advances in Neural Information Processing Systems*. 33. köt. 2020, 1877–1901. old.

[27] B. T. Willard és R. Louf. *Efficient Guided Generation for Large Language Models*. arXiv:2307.09702. 2023.

[28] P. Lewis, E. Perez, A. Piktus és tsai. „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. *Advances in Neural Information Processing Systems*. 33. köt. 2020, 9459–9474. old.

[29] Y. Gao, Y. Xiong, X. Gao és tsai. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2023. arXiv: 2312.10997.

[30] S. Robertson és H. Zaragoza. „The Probabilistic Relevance Framework: BM25 and Beyond”. *Foundations and Trends in Information Retrieval* 3.4 (2009), 333–389. old.

[31] V. Karpukhin, B. Oğuz, S. Min és tsai. „Dense Passage Retrieval for Open-Domain Question Answering”. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, 6769–6781. old.

[32] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder és F. Wei. *Multilingual E5 Text Embeddings: A Technical Report*. 2024. arXiv: 2402.05672.

[33] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava és I. Gurevych. „BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models”. *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. 2021.

[34] G. V. Cormack, C. L. A. Clarke és S. Buettcher. „Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009, 758–759. old.

[35] L. Gao, X. Ma, J. Lin és J. Callan. „Precise Zero-Shot Dense Retrieval without Relevance Labels”. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. 2023, 1762–1777. old.

[36] R. Qu, R. Tu és F. Bao. *Is Semantic Chunking Worth the Computational Cost?* 2024. arXiv: 2410.13070.

[37] T. Gao, H. Yen, J. Yu és D. Chen. „Enabling Large Language Models to Generate Text with Citations”. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, 6465–6488. old.

[38] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das és tsai. „Measuring Attribution in Natural Language Generation Models”. *Computational Linguistics* 49.4 (2023), 777–840. old.

[39] D. Edge, H. Trinh, N. Cheng és tsai. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. 2024. arXiv: 2404.16130.

[40] C.-P. Hsieh, S. Sun, S. Kriman és tsai. „RULER: What’s the Real Context Size of Your Long-Context Language Models?”: *Conference on Language Modeling (COLM)*. 2024.

[41] Z. Li, C. Li, M. Zhang, Q. Mei és M. Bendersky. „Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach”. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 2024.

[42] J. Wei, X. Wang, D. Schuurmans és tsai. „Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. *Advances in Neural Information Processing Systems*. 35. köt. 2022.

[43] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang és tsai. „Least-to-Most Prompting Enables Complex Reasoning in Large Language Models”. *International Conference on Learning Representations (ICLR)*. 2023.

[44] L. Wang, W. Xu, Y. Lan és tsai. „Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models”. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. 2023, 2609–2634. old.

[45] T. Wu, M. Terry és C. J. Cai. „AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts”. *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 2022.

[46] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe és tsai. „Self-Refine: Iterative Refinement with Self-Feedback”. *Advances in Neural Information Processing Systems*. 36. köt. 2023.

[47] S. Dhuliawala, M. Komeili, J. Xu és tsai. „Chain-of-Verification Reduces Hallucination in Large Language Models”. *Findings of the Association for Computational Linguistics: ACL 2024*. 2024.

[48] S. Min, K. Krishna, X. Lyu és tsai. „FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation”. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, 12076–12100. old.

[49] P. Laban, T. Schnabel, P. N. Bennett és M. A. Hearst. „SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization”. *Transactions of the Association for Computational Linguistics* 10 (2022), 163–177. old.

[50] C. Snell, J. Lee, K. Xu és A. Kumar. „Scaling LLM Test-Time Compute Optimally Can be More Effective than Scaling Parameters for Reasoning”. *International Conference on Learning Representations (ICLR)*. 2025.

[51] M. Strathern. „‘Improving Ratings’: Audit in the British University System”. *European Review* 5.3 (1997), 305–321. old.

[52] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv: 2412.15115.

[53] A. Grattafiori és tsai. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783.

[54] Gemma Team. *Gemma 3 Technical Report*. 2025. arXiv: 2503.19786.

[55] D. M. Nemeskey. „Introducing huBERT”. *XVII. Magyar Számítógépes Nyelvészeti Konferencia (MSZNY 2021)*. Szeged, 2021, 3–14. old.

[56] NYTK – HUN-REN Nyelvtudományi Kutatóközpont. *PULI-LlumiX-Llama-3.1: magyar–angol nyelvi modell (8B), a Llama 3.1 továbbtanításával*. https://huggingface.co/NYTK/PULI-LlumiX-Llama-3.1. Modellkártya. 2024.

[57] E. Frantar, S. Ashkboos, T. Hoefler és D. Alistarh. „GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers”. *International Conference on Learning Representations (ICLR)*. 2023.

[58] G. Gerganov és tsai. *llama.cpp: LLM inference in C/C++*. https://github.com/ggml-org/llama.cpp. Szoftver, GGUF formátum és K-kvantálás. 2023.

[59] Ollama. *Ollama: Get up and running with large language models locally*. https://github.com/ollama/ollama. Szoftver. 2024.

[60] S. Li, X. Ning, L. Wang és tsai. „Evaluating Quantized Large Language Models”. *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 235. köt. PMLR. 2024, 28480–28524. old.

[61] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang és tsai. „Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*. 36. köt. 2023.

[62] C.-H. Chiang és H.-y. Lee. „Can Large Language Models Be an Alternative to Human Evaluations?": *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. 2023, 15607–15631. old.

[63] S. Es, J. James, L. Espinosa-Anke és S. Schockaert. „RAGAs: Automated Evaluation of Retrieval Augmented Generation". *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. 2024, 150–158. old.

[64] P. Wang, L. Li, L. Chen és tsai. „Large Language Models are not Fair Evaluators". *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024, 9440–9450. old.

[65] A. Panickssery, S. R. Bowman és S. Feng. „LLM Evaluators Recognize and Favor Their Own Generations". *Advances in Neural Information Processing Systems*. 37. köt. 2024.

[66] OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: 2508.10925.

[67] J. Cohen. „A Coefficient of Agreement for Nominal Scales". *Educational and Psychological Measurement* 20.1 (1960), 37–46. old.

[68] K. Krippendorff. *Content Analysis: An Introduction to Its Methodology*. 2. kiad. Thousand Oaks, CA: Sage, 2004.

[69] J. R. Landis és G. G. Koch. „The Measurement of Observer Agreement for Categorical Data". *Biometrics* 33.1 (1977), 159–174. old.

[70] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto és tsai. „Accounting for Variance in Machine Learning Benchmarks". *Proceedings of Machine Learning and Systems (MLSys)*. 2021.

[71] R. Dror, G. Baumer, S. Shlomov és R. Reichart. „The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing". *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*. 2018, 1383–1392. old.

[72] S. Holm. „A Simple Sequentially Rejective Multiple Test Procedure". *Scandinavian Journal of Statistics* 6.2 (1979), 65–70. old.

[73] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. Lopez de Lacalle és E. Agirre. „NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for each Benchmark". *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023.

[74] R. Parasuraman és D. H. Manzey. „Complacency and Bias in Human Use of Automation: An Attentional Integration". *Human Factors* 52.3 (2010), 381–410. old.

[75] N. Carlini, F. Tramèr, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee és tsai. „Extracting Training Data from Large Language Models". *30th USENIX Security Symposium*. 2021, 2633–2650. old.

[76] Európai Parlament és a Tanács. *Az Európai Parlament és a Tanács (EU) 2024/1689 rendelete a mesterséges intelligenciára vonatkozó harmonizált szabályok megállapításáról (mesterséges intelligenciáról szóló rendelet)*. Az Európai Unió Hivatalos Lapja, L sorozat. 2024.

[77] F. Miao és W. Holmes. *Guidance for Generative AI in Education and Research*. Paris: UNESCO, 2023.

[78] K. Ahuja, H. Diddee, R. Hada, M. Ochieng, K. Ramesh, P. Jain és tsai. „MEGA: Multilingual Evaluation of Generative AI". *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023.

[79] N. Ligeti-Nagy, G. Ferenczi, E. Héja és tsai. „HuLU: Hungarian Language Understanding Benchmark Kit". *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. 2024, 8360–8371. old.

[80] A. Novák, B. Novák, D. Zombori, G. Szabó, Z. Szántó és R. Farkas. „A Question Answering Benchmark Database for Hungarian". *Proceedings of the 17th Linguistic Annotation Workshop (LAW-XVII)*. 2023, 188–198. old.

[81] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: SELECT – The Locking Clause (FOR UPDATE SKIP LOCKED)*. `https://www.postgresql.org/docs/16/sql-select.html`. 2024.

[82] Qdrant. *Qdrant: Vector Similarity Search Engine*. `https://github.com/qdrant/qdrant`. Szoftver. 2024.

[83] M. L. Balinski és H. P. Young. *Fair Representation: Meeting the Ideal of One Man, One Vote*. 2. kiad. Washington, D.C.: Brookings Institution Press, 2001.

[84] B. Efron és R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

[85] F. Wilcoxon. „Individual Comparisons by Ranking Methods". *Biometrics Bulletin* 1.6 (1945), 80–83. old.

[86] D. S. Kerby. „The Simple Difference Formula: An Approach to Teaching Nonparametric Correlation". *Comprehensive Psychology* 3 (2014).

Ábrák jegyzéke

3.1. A MIMIR szolgáltatásai és adatfolyama. A feltöltött tartalmat a feldolgozó szolgáltatások csak a feladat élettartama alatt, memóriában tartják; a nyelvi modell az egyetlen GPU-t használó összetevő, külső modellt csak kikapcsolt `LOCAL_ONLY` kapcsoló mellett hív a rendszer. 14
3.2. A naiv RAG és a MIMIR tervezett generálási folyamata. A terv a vizsga szerkezetét még az első kérdés megírása előtt rögzíti, és minden kérdést a saját, hivatkozott kontextusával szemben ellenőriz. 16
3.3. Az ellenőrző lánc visszacsatolással. Az olcsó szabályalapú ellenőrzések előbb futnak; a hiba szövege a következő generálási kísérlet promptjába kerül. 20
3.4. A webes szerkesztő egy generált feleletválasztós kérdéssel: a megjelölt helyes válasz, alatta a hivatkozott forrásrészlet („1 forrás”, fájlnév és oldalszám); jobb felül a típusválasztó, az újragenerálás, az áthelyezés és a törlés gombja. 21

4.1. Egy teszt lépései: előkészítés a szolgáltatásokon, generálás a konfiguráció szerint, automatikus mutatók, bírói pontozás és összesítés. 22
4.2. A konfigurációk egytényezős fája: minden él egyetlen tényező megváltoztatása. 24

5.1. A minőségi index konfigurációnként és generátormodellenként, 95 %-os bootstrap-intervallummal. A Qwen2.5-7B az előzetes vizsgálat hat dokumentumán, a másik két modell a fő vizsgálat 30 dokumentumán. A szerveres konfigurációk csak az előzetes vizsgálatban futottak. 33
5.2. A minőségi index a módszer lépései szerint (naiv RAG $\rightarrow$ vizsgaterv $\rightarrow$ ellenőrző lánc) és a teljes dokumentumos prompt (szaggatott vonal) a három helyi modellel. Minél gyengébb a modell, annál nagyobb a módszer haszna. 34
5.3. A minőségi index dokumentumonkénti eloszlása a fő vizsgálatban (pontok: egy-egy dokumentum; doboz: interkvartilis tartomány, vastag vonal: medián). A pontozott vonal a 0,70-es szint. 35
5.4. A minőségi index dokumentumforrásonként a fő vizsgálat négy fő konfigurációjában. Zárójelben a dokumentumok száma. 36
5.5. Minőség és futási idő. Az egyhívásos konfigurációk gyorsak, de minőségük a modelltől és a darabolástól függően erősen ingadozik; a tervezett konfigurációk lassabbak, de minden modellnél a legjobbak között vannak. 39

Táblázatok jegyzéke

2.1. A kérdésgenerálási megközelítések összevetése a P1–P4 problémák szempontjából (✓: kezeli, ○: részben, –: nem kezeli). A táblázat azt mutatja, hogy egy megközelítés foglalkozik-e az adott szemponttal, nem azt, hogy milyen jól. . . 12

3.1. Tervezési célok: a dokumentum csevegő nyelvi modellnek való átadása és a MIMIR összevetése. 13
3.2. Adatfajták életciklusa a MIMIR rendszerben. 15
3.3. Becsült GPU-memóriaigény 8 192 tokenes kontextusnál, Q4_K_M kvantálással és $M_0 \approx 0{,}6$ GB többlettel. A súlyok a GGUF-fájlok mérete, m_{kv} a publikált architektúrákból számolt érték. A Gemma 3 12B sorában mért érték áll (lásd a szöveget). 16
3.4. A nehézségi szintekhez rendelt Bloom-eloszlások és egy 10 kérdéses vizsga kiosztása. 18
3.5. Az ellenőrző lánc lépései végrehajtási sorrendben, a hiba esetén adott visszajelzéssel. 19

4.1. Az `eval-v1` adatkészlet és a fő vizsgálatban használt része. R/K/H: rövid ($\leq 6\,000$), közepes (6 000–25 000) és hosszú (25 000–90 000 karakteres) dokumentumok. 23
4.2. A kísérleti konfigurációk. Ref.: amelytől a konfiguráció egyetlen tényezőben eltér; Magok: dokumentumonkénti ismétlések száma az előzetes és a fő vizsgálatban (a pilotban mindenhol 3); Vizsg.: P = pilot, Ev = előzetes vizsgálat, F = fő vizsgálat. 24
4.3. A három vizsgálat beállításai. 25

5.1. Az előzetes vizsgálat fő eredményei (Qwen2.5-7B, hat dokumentum; az egyhívásos konfigurációk három, a tervezettek egy maggal). Q: minőségi index 95 %-os bootstrap-intervallummal; F, G, A, V: formai megfelelés, alátámasztottság, vak megoldhatóság, disztraktor-validitás; Lef.: lefedettség (%). *Az E4 és az E4b hat angol dokumentumon futott, amelyek közül három közös a többiekével. . . 31
5.2. A fő vizsgálat eredményei (30 dokumentum). Q: minőségi index 95 %-os bootstrap-intervallummal; F, G, V: formai megfelelés, alátámasztottság, disztraktor-validitás (%); Min.: a leggyengébb dokumentum Q-ja; Perc, Hívás: medián futási idő és modellhívás vizsgánként. *Az E2x-ben a második modell ellenőrző hívásai hibával tértek vissza, így csak a szabályalapú ellenőrzés futott. 32
5.3. Páros Wilcoxon-próbák a dokumentumonkénti minőségi indexen (fő vizsgálat, 30 dokumentum). Δ: az új és a referencia-konfiguráció különbsége; r_{rb}: rangbiszeriális hatásméret; Jobb/rosszabb: hány dokumentumon javított, illetve rontott az új konfiguráció; $\tilde{p}$: a hat összevetésre Holm-korrigált p-érték. Félkövér: $\tilde{p} < 0{,}05$. 34
5.4. Az E2 ellenőrző láncának működése modellenként. Első kísérlet: az első próbálkozásra elfogadott kérdéshelyek aránya; Átengedett: az ellenőrzöttként elfogadott kérdések aránya; pontosság, felidézés és Cohen-κ a bíró alátámasztottsági ítéletéhez képest (csak a bírált kérdéseken). Az elutasítási okok az összes kísérletre vonatkoznak. 37

5.5. A helyi modellek erőforrásigénye. Modell: az Ollama által jelentett foglalás 8 192 tokenes kontextusnál (MiB); Kártya: a szükséges fogyasztói GPU az asztali környezet 1,6–1,9 GiB-os foglalása mellett (a pilot mérése); Perc: medián futási idő vizsgánként RTX A4500-on. 38

6.1. A hipotézisek és a vizsgálatok bizonyítéka. Fő: a 30 dokumentumos fő vizsgálat (PULI-LlumiX 8B és Gemma 3 12B); Előzetes: a hat dokumentumos vizsgálat (Qwen2.5-7B és a szervermodell). 40

C.1. A generálási folyamat paraméterei és alapértékei az előzetes és a fő vizsgálatban. 59

A. Promptok

Ez a függelék a generálási folyamatok promptjainak lényegi sorait közli; a kihagyásokat [...] jelöli, a kapcsos zárójelben álló részek helyére a program illeszti be az aktuális értéket. A belső lépések (tervező, gráf, ellenőrző) promptjai angol nyelvűek, mert a kis helyi modellek ezeket megbízhatóbban követik; a generátor promptja magyar, és megismétli az éles prompt szabályait, így az E0 és az E1–E3 konfigurációk szerkezetben, nem szabályokban különböznek. A teljes szövegek: `services/bifrost/prompts.py` és `tests/eval/mimir_eval/blueprint/prompts.py`.

A.1. A naiv RAG (E0) és a teljes dokumentumos prompt (B-doc)

```
Te egy kiemelkedő tudású oktatásmódszertani szakértő és professzionális vizsgakészítő vagy.
KÖTELEZŐ SZABÁLYOK, AMIKET SZIGORÚAN BE KELL TARTANOD:
1. ZÉRÓ HALLUCINÁCIÓ: KIZÁRÓLAG a megadott KONTEXTUS alapján dolgozz! [...]
2. FELHASZNÁLÓI UTASÍTÁS KÖVETÉSE: [...] Pontosan a kért mennyiséget és típust generáld le!
3. DISZTRAKTOROK: Feleletválasztós (mcq) kérdés esetén 1 helyes és 3 hihető, de helytelen
↪  válasz.
4. NO LATEX: Szigorúan TILOS LaTeX formázást vagy dollárjeleket ($) használni!
5. META-REFERENCIA TILALOM: Szigorúan TILOS a dokumentum szerkezetére kérdezni. [...]
KONTEXTUS:
{context_text}            (E0: a 3 legközelebbi darab; B-doc: a teljes dokumentum)
FELADAT (A felhasználó pontos kérése):
"Készíts pontosan 10 darab vizsgakérdést, típus: feleletválasztós (mcq). Nehézségi szint:
↪  ..."
KIMENETI FORMÁTUM: [...] kizárólag egy érvényes JSON objektum:
{"title": "Mimir AI Vizsga", "format": "pdf", "questions": [
  {"type": "mcq", "text": "...?", "answers": [{"text": "...", "is_correct": true}, ...]}]}
```

A.2. A tervező

```
[planner] You design exam blueprints. Reply with one valid JSON object only.
Below is a document split into numbered chunks (C0, C1, ...). Some chunks are shortened.
{overview}                (darabonként az eleje, összesen legfeljebb 12 000 karakter)
List the {n+3} most important, clearly distinct concepts, facts or ideas a student should be
tested on. Cover the whole document: every major part should have at least one concept.
↪  Prefer
central ideas over trivia. [...]
JSON shape: {"concepts": [{"concept": "short name", "description": "one sentence: what
↪  exactly
  should be tested", "chunk_ids": ["C0"], "importance": 3}]}
importance: 3 = central, 2 = important, 1 = detail.
```

A.3. A generátor

```
[generator] Te egy kiemelkedő tudású oktatásmódszertani szakértő és professzionális
↪  vizsgakészítő
vagy. Kizárólag érvényes JSON formátumban válaszolj, markdown formázás nélkül!
Készíts PONTOSAN EGY vizsgakérdést az alábbi terv szerint.
```

```
TERV:
- Fogalom: {concept}       - Mit kérdezzen: {description}
- Kérdéstípus: feleletválasztós ("mcq"): pontosan 4 válasz, ebből pontosan 1 helyes és 3
 ↪  hihető,
  de a kontextus szerint egyértelműen helytelen disztraktor; [...] hossza és formája legyen
   ↪  hasonló.
- Bloom-szint: {pl. Megértés (magyarázat, összehasonlítás, saját szavakkal)}
- Vizsga nehézsége: {difficulty}    - Nyelv: [...] {magyar} nyelven írd.
KÖTELEZŐ SZABÁLYOK:
1. ZÉRÓ HALLUCINÁCIÓ: KIZÁRÓLAG a KONTEXTUS alapján dolgozz. [...]
2. HIVATKOZÁS: a "citations" mezőbe írd azoknak a részleteknek az azonosítóját (pl. "C3"),
   amelyek igazolják a helyes választ.
3. NO LATEX. [...]  4. META-REFERENCIA TILALOM: [...]
5. NE ISMÉTELD a már elkészült kérdéseket: {existing}
[DISZTRAKTOR-ÖTLETEK (rokon fogalmak a dokumentumból) ...]          (csak E3)
[AZ ELŐZŐ PRÓBÁLKOZÁS HIBÁS VOLT, JAVÍTSD: {feedback}]             (csak újrapróbálkozáskor)
KONTEXTUS:
[C3] ...
[C1] ...
KIMENETI FORMÁTUM (csak ez a JSON objektum):
{"type": "mcq", "text": "A kérdés szövege?", "answers": [...], "citations": ["C0"],
 "bloom": "understand", "explanation": "egy mondat: miért ez a helyes válasz"}
```

A.4. A javító hívás

```
[repair] Te egy gondos vizsgakészítő vagy, aki hibás formájú kérdéseket javít. [...]
Az alábbi feleletválasztós kérdés formája hibás. Javítsd ki úgy, hogy PONTOSAN 4 válasz
 ↪  legyen,
ebből PONTOSAN 1 helyes; a hiányzó rossz válaszokat a kontextus alapján egyértelműen hibás,
 ↪  de
hihető állításokkal pótold. A kérdés tartalmán és a helyes válaszon ne változtass [...]. A
 ↪  kérdés ne
tartalmazza a helyes választ, és kérdőjellel végződjön.
KÉRDÉS: {text}   VÁLASZOK: {options}   KONTEXTUS: {chunks}   KIMENETI FORMÁTUM: [...]
```

A.5. Az ellenőrző

```
[verifier] You check exam questions against source text. Reply with one valid JSON object
 ↪  only.
--- alátámasztottság és disztraktorok ---
SOURCE: """{a hivatkozott darabok}"""
QUESTION (mcq): {text}
A. ...   <-- marked correct
B. ...  [...]
Check strictly against the SOURCE only:
1. key_supported: is the marked-correct answer clearly supported by the SOURCE?
2. For every option NOT marked correct: is it clearly wrong according to the SOURCE? [...]
3. ambiguous: could a careful reader argue that more than one option is correct?
4. problems: one short sentence describing any problem ("" if none).
JSON shape: {"key_supported": true, "options_wrong": [true, true, true], "ambiguous": false,
             "problems": ""}
--- vak megoldás ---
SOURCE: """{a hivatkozott darabok}"""
Answer the question using only the SOURCE. Exactly one option is correct.
{text}
```

```
A. ...  B. ...  C. ...  D. ...        (véletlen permutáció)
JSON shape: {"answer": "A"}
```

A.6. A fogalomgráf (E3)

```
[graph] You extract knowledge graphs from teaching material. Reply with one valid JSON object
↪  only.
Extract the knowledge in these chunks as a small concept graph. Chunk ids are in brackets.
{chunks_block}            (legfeljebb 3 000 karakteres köteg)
Rules:
- concepts: the key terms, entities, processes or ideas (max 8 per chunk). "parent" is the
↪  broader
  category the concept belongs to (e.g. "T-sejt" -> "limfocita"), or "" if none. [...]
- facts: short atomic statements from the text (max 6 per chunk), [...]
- relations: typed links between concepts: is_a, part_of, causes, contrasts_with, used_for,
  related_to. [...]
JSON shape: {"concepts": [...], "facts": [...], "relations": [...]}
```

B. A bíró és az oktatók rubrikája

A rubrika szövege a bíró és az oktatói értékelőlap számára azonos (a bíró az angol, az oktatók a magyar változatot kapják).

Helyesség

A megjelölt válasz a forrás szerint helyes, és más opció nem helyes. 1 = hibás kulcs vagy két helyes opció; 3 = helyes, de pontatlan; 5 = teljesen helyes, egyértelmű.

Érthetőség

A kérdés önmagában érthető, nyelvtanilag helyes, egyértelmű, nem hivatkozik „a szövegre” vagy a dokumentum szerkezetére. 1 = zavaros; 3 = nehezen érthető; 5 = vizsgakész megfogalmazás.

Disztraktorok

A rossz válaszok a forrás szerint egyértelműen rosszak, de egy felkészületlen diák számára hihetők (azonos téma, hasonló hossz és forma). 1 = abszurd vagy szintén helyes; 3 = részben hihető; 5 = mind hihető és rossz. Kifejtős kérdésnél üresen marad.

Szintillesztés

A kognitív szint megfelel a kért nehézségnek (könnyű: felidézés/megértés; közepes: alkalmazás/megértés; nehéz: elemzés/értékelés). 1 = egyértelműen rossz szint; 3 = határeset; 5 = illik.

Felhasználnám

Beraknád ezt a kérdést egy valódi dolgozatba legfeljebb apró javítással? (igen/nem)

A bíró áttekintő hívása ezen felül kéri a kulcsot igazoló mondat szó szerinti idézését (`key_evidence` mező), disztraktoronként az `is_wrong` és az `on_topic` ítéletet, a kérdés Bloom-szintjét és egy-két mondatos indoklást.

C. Paraméterek és reprodukálás

C.1. A generálási folyamat paraméterei

C.1. táblázat. A generálási folyamat paraméterei és alapértékei az előzetes és a fő vizsgálatban.

Paraméter	Alapérték	Megjegyzés
darabolás	percentilis, 85	célméret 1 000 karakter, puha korlát; E5b: átlag $+1{,}5$ szórás; E5c: rögzített 800 karakter
visszakeresett darabok (k)	3	E0: a teljes vizsgához; tervezett folyamat: kérdéshelyenként
keresés módja	sűrű	E2h: BM25 + sűrű, RRF ($c = 60$), 6 karakteres előtagtövesítés
áttekintés kerete	12 000 karakter	a tervező ennyit lát; darabonként legalább 120 karakter
fogalmak száma	$n + 3$	3 tartalékfogalom
kérdéshely-kontextus kerete	6 000 karakter	+ 2 darab, ha a kulcs nem igazolt
újrapróbálkozás (r)	1	pilot: 2; utána 1 tartalékfogalom
ellenőrző lánc	E1: ki; E2, E3: be	vak megoldás: be (E2f: ki)
szivárgási szabályok, javító hívás	be	a pilot után bevezetve
duplikátumküszöb	0,9	koszinusz-hasonlóság (E5-beágyazás)
fogalomgráf (E3)	3 000 karakteres köteg	összevonás 0,92 hasonlóság felett
teljes dokumentum (B-doc)	30 000 / 80 000 karakter	helyi / szerver; helyben 16 384 tokenes kontextus
generátor	Q4_K_M, $T = 0{,}2$	8 192 tokenes kontextus; `num_predict` 4 096 (fő vizsgálat)
bíró	gpt-oss-120b, $T = 0$	„low” gondolkodás; forrás legfeljebb 24 000 karakter

Minden konfiguráció egy YAML-fájl a `tests/eval/configs` könyvtárban, amely csak az alapértékektől (`mimir_eval/config.py`) eltérő kulcsokat adja meg. Az egytényezős terv a fájlok szintjén is látható: az E2 és az E3 konfiguráció például egyetlen kulcsban (`graph`) különbözik.

```
# E2 - E1 + verifier chain: schema, meta-reference, citations, grounding + distractor check,
# blind answer test; max 1 retry per slot with feedback, then 1 replacement.
name: E2
seeds: [1]
```

```
exam:
  n_questions: 10
  types: [mcq]
pipeline:
  kind: blueprint        # retrieval: dense, retrieval_k: 3
  verifier: true
  blind_test: true
  max_retries: 1
  graph: false           # E3: true
chunking:
  method: percentile     # threshold: 85, target_size: 1000
generator:
  provider: ollama
  model: "hf.co/mradermacher/PULI-LlumiX-Llama-3.1-GGUF:Q4_K_M"
  temperature: 0.2
  num_ctx: 8192
```

C.2. A reprodukálás lépései

A nyers eredmények a tárolóban vannak: a fő vizsgálaté a `tests/eval/results`, az előzetes vizsgálaté a `tests/eval/results_qwen_2026-09`, a pilotté a `tests/eval/results_pilot` könyvtárban (konfigurációnként egy-egy futáskönyvtár `run.json`, `exams.jsonl`, `questions.jsonl` és `exam_metrics.csv` fájlokkal). A dolgozat minden száma egyetlen szkriptből származik: a `python tdk/elemzes/elemzes.py` parancs a három futáskészletből a `tdk/elemzes/szamok.md` (olvasható) és `szamok.json` (gépi) fájlt állítja elő.

```
# 1. Szolgáltatások és helyi modellek
docker-compose up -d --build
ollama pull hf.co/mradermacher/PULI-LlumiX-Llama-3.1-GGUF:Q4_K_M
ollama pull gemma3:12b
ollama pull llama3.1:8b                      # az E2x ellenőrzője

# 2. Kiértékelő környezet és ellenőrzés
cd tests/eval
pip install -r requirements.txt
python -m pytest tests_unit -q               # a keretrendszer saját tesztjei
python -m mimir_eval check configs/e0_naive_local.yaml --judge-model gpt-oss:120b

# 3. Egy konfiguráció: futtatás, pontozás
python -m mimir_eval run   configs/e2_blueprint_verifier.yaml
python -m mimir_eval score results/E2_<időbélyeg> --judge-model gpt-oss:120b

# 4. A teljes mátrix (Windows PowerShell); megszakítás után folytatható
./run_all.ps1                                # PULI-LlumiX 8B (alapértelmezett generátor)
./run_all.ps1 -Model gemma3:12b              # a helyi konfigurációk a második generátorral

# 5. A dolgozat számai (a tároló gyökeréből)
python tdk/elemzes/elemzes.py                # -> tdk/elemzes/szamok.md, szamok.json
```

A `-Model` kapcsoló a helyi konfigurációkat egy másik Ollama-modellel futtatja új néven (például E2 → `E2-gemma3-12b`), így a két generátor eredményei egymás mellett maradnak; a szerverkonfigurációkat (B-doc-S, E4, E4b) ilyenkor kihagyja. Az előzetes vizsgálat ugyanígy,

`-Model qwen2.5:7b` kapcsolóval ismételhető meg. A bíró a GenAI-szerver kulcsát a tároló `.env` fájljából (`OE_GENAI_API_KEY`) olvassa; válaszai a futáskönyvtár `judge_cache.jsonl` fájljába kerülnek, így az újrapontozás nem igényel új bírói hívást.